



POLITECNICO DI TORINO

MASTER'S DEGREE IN CYBERSECURITY ENGINEERING

Design and Implementation of an eBPF-based Security Monitoring Tool

Supervisor:

Prof. Riccardo Sisto

Candidate:

Simone Romano

Company Tutors:

Dr. David Soldani

Dr. Vincenzo Cosi

Academic Year 2025-2026

This thesis presents the design and implementation of a security monitoring tool that collects and analyzes process-related information directly from the operating system kernel using eBPF (extended Berkeley Packet Filter). By leveraging kernel-level instrumentation, the system captures detailed process events with high accuracy and minimal overhead, overcoming the limitations of traditional user-space monitoring approaches.

The proposed tool adopts a modular and efficient architecture to ensure scalability and real-time data collection. The gathered information is structured to support security analysis, enabling the detection of anomalous behaviors such as unauthorized executions and privilege escalations.

This work demonstrates how kernel-derived process data, combined with eBPF technology, can enhance system visibility and provide a solid foundation for advanced security monitoring and threat detection.

Acknowledgements

Only in the printed version

Contents

Acronyms	1
1 Introduction	2
1.1 Context and Motivation	2
1.2 Problem Statement	3
1.3 Objectives and Thesis Structure	5
2 eBPF Architecture	7
2.1 Linux Kernel Observability	7
2.2 eBPF Fundamentals	11
2.2.1 Execution Model, Verifier and JIT Compilation	11
2.2.2 Object Files, Loading and Attachment Lifecycle	13
2.2.3 Hook Selection and Event Semantics	14
2.2.4 Maps, Helpers and Kernel-to-Userspace Transport	15
2.2.5 BTF and CO-RE Portability	17
2.3 Related eBPF Runtime Security Tools	18
2.3.1 Tracee	18
2.3.1.1 Architecture and Event Pipeline	19
2.3.1.2 Unified Event Model	20
2.3.1.3 Policies and Event Selection	20
2.3.1.4 Detectors and Derived Events	21

CONTENTS

2.3.1.5	Output and Operational Context	22
2.3.1.6	Portability and Security Boundaries	22
2.3.1.7	Relationship to the Proposed Work	23
2.3.2	Other Relevant eBPF Security Tools	24
2.3.3	Comparative Summary	25
2.4	Positioning of the Proposed Work	25
3	Requirements and System Design	27
3.1	Design Goals and Constraints	28
3.1.1	Target Environment and Compatibility Assumptions .	28
3.1.2	Functional Requirements	29
3.1.3	Non-Functional and Operational Requirements	30
3.2	System Architecture	30
3.2.1	Kernel-Space and User-Space Responsibilities	31
3.2.2	End-to-End Event Lifecycle	32
3.3	Kernel-Space Collection Design	34
3.3.1	Hook Selection and Logical Events	34
3.3.2	Early Event Selection and Kernel-Side Filtering	35
3.4	Event Contract and Transport Design	36
3.4.1	Stable Binary Event Contract	37
3.4.2	Perf-Buffer Transport and Failure Boundaries	38
3.5	User-Space Runtime Design	39
3.5.1	Loading, Attachment and Runtime Lifecycle	39
3.5.2	Decoding, Normalisation and Event Registration	40
3.5.3	Event Output, Alert Output and Operational Logging	41
3.6	Policy and Detection Architecture	42
3.6.1	Policy-Based Event Scope	43
3.6.2	Point and Collective Detectors	44
3.6.3	Bounded Local Correlation	44

CONTENTS

3.6.4	Alert Evidence and MITRE ATT&CK Metadata . . .	45
3.7	Design Decisions and Scope Boundaries	46
4	Kernel-Space Event Collection Implementation	48
4.1	Build Toolchain and Project Organisation	48
4.2	eBPF Program Implementation	49
4.2.1	Shared Event-Construction Framework	49
4.2.2	Process Lifecycle and Executable Transitions	52
4.2.2.1	Process Creation and Lifecycle	52
4.2.2.2	Executable Transitions	54
4.2.3	Identity, Credentials, Process Control and Limits . . .	56
4.2.3.1	Identity and Credentials	56
4.2.3.2	Process Control and Limits	59
4.2.4	File and Filesystem Security	61
4.2.5	Memory, Namespace and Cgroup Transitions	65
4.2.6	Kernel Integrity and Kernel-Facing Activity	68
4.3	Binary Event Contract and Kernel Transport	71
4.4	Compatibility and Verifier Challenges	72
4.5	Kernel-Space Implementation Summary	73
5	User-Space Runtime and Detection Implementation	74
5.1	Loader, Probe Registry and Attachment Lifecycle	74
5.2	User-Space Decoder and Event Registry	77
5.3	User-Space Event Selection and Admission	79
5.4	Policy and Detector Engine	81
5.4.1	YAML Policy and Detector Definitions	81
5.4.2	Point and Collective Detection	81
5.4.3	Local Correlation Strategies, State Bounds and Alert Deduplication	82

CONTENTS

5.4.4	MITRE ATT&CK Metadata Propagation	85
5.5	Output and Structured Logging	85
5.6	Containerisation and Deployment Considerations	86
5.7	User-Space Implementation Summary	87
A	Appendix A	88
B	Appendix B	89
	Bibliography	90

List of Figures

2.1	High-level eBPF execution workflow	12
2.2	Complementary evidence exposed by different observation points along a kernel operation.	15
2.3	Tracee high-level architecture	19
3.1	Kernel space and userspace architecture	31
3.2	Responsibility boundary and end-to-end flow of the proposed monitoring system.	33
3.3	Conceptual relationship between the binary event record, the user-space event registry, and the resulting normalised event.	37
3.4	User-space runtime lifecycle and failure boundaries	40
4.1	Execution monitoring: attempts and outcomes	54
4.2	Real, Effective, Saved and Filesystem UIDs	57
4.3	Memory and isolation transition monitoring	65

List of Tables

2.1	Observation sources for eBPF runtime monitoring	10
2.2	Comparison of eBPF event-transport mechanisms	17
2.3	Architectural summary of the runtime security tools considered in this chapter.	25
3.1	Functional requirements of the proposed system.	29
3.2	Non-functional and operational requirements.	30
3.3	Distinction between physical hooks, producer and support probes, and logical events.	35
5.1	Decoder admission and permissiveness boundaries.	79
5.2	Local-correlation and alert-deduplication bounds.	84
5.3	User-space output and diagnostic boundaries.	86

Acronyms

eBPF	extended Berkeley Packet Filter
CPU	central processing unit
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
LSM	Linux Security Module
JIT	just-in-time
BTF	BPF Type Format
CO-RE	Compile Once - Run Everywhere
ELF	Executable and Linkable Format

Chapter 1

Introduction

1.1 Context and Motivation

Modern Linux systems execute applications, containers, and infrastructure services whose security-relevant behaviour changes continuously at runtime. Static inspection can describe software before execution, while application logs expose only the information that applications choose to report. Neither source alone provides a complete account of operations such as process creation, credential changes, file access, memory protection changes, namespace activity, or interactions with kernel security mechanisms. Runtime security monitoring therefore requires an observation point that is close enough to the operating system to capture these actions as they occur.

The Linux kernel is a valuable observation layer because it mediates processes, files, credentials, memory, and other shared resources. Kernel-level telemetry can reveal behaviour that is distributed across several applications or omitted from their logs. This visibility is particularly relevant in containerised environments, where workloads remain isolated at the application level but share the host kernel. At the same time, kernel instrumentation is security-sensitive: excessive privileges or unsafe extensions may increase the attack surface, especially across container boundaries [20]. A monitoring design must consequently balance visibility, compatibility, operational cost, and the risks associated with privileged execution.

Runtime security tools increasingly use eBPF to obtain this visibility. Existing systems demonstrate that eBPF can support process, file, network, and security telemetry in cloud-native environments, although their archi-

textures and performance characteristics differ [21]. This thesis focuses on the process and host-security side of that design space.

eBPF provides a controlled runtime inside the Linux kernel. User-space software loads constrained programs that can be attached to supported processes, networking, and security hooks. Before a program is accepted, the kernel verifier analyses its control flow and use of memory, registers, pointers, and helper functions through static analysis. Accepted programs can then collect or transform data when the corresponding hook is reached [18, 22, 23].

This execution model is well suited to runtime monitoring for three reasons. First, collection occurs at the point where the kernel processes the operation, reducing dependence on application-specific instrumentation. Second, eBPF maps and event buffers provide structured communication between kernel and user space. Third, programs can reject irrelevant activity before transporting it to user space, which creates an opportunity to control processing overhead. These properties do not make an eBPF tool automatically safe, portable, or efficient: verifier behaviour, available program types, helper functions, kernel data structures, and attachment mechanisms vary across kernel versions. They instead provide the foundation on which a target-aware monitoring pipeline can be built.

The work was developed in an industrial collaboration and targets Rocky Linux 8.10 with the kernel `4.18.0-553.109.1.el8_10.x86_64`. This environment is important because it combines a long-lived enterprise kernel line with backported functionality. A feature exposed by a newer upstream kernel cannot be assumed to exist, or to behave identically, on this target [27]. Compatibility is therefore treated as an architectural constraint.

Within this target environment, the prototype is designed as a host-level runtime monitor. It collects and processes security-relevant events on the monitored system and performs bounded local detection over individual events and short event sequences.

1.2 Problem Statement

The central problem addressed by this thesis is how to convert heterogeneous kernel observations into useful and readable security information on a constrained enterprise Linux target. Capturing an event is only the first step. A practical pipeline must assign a stable identity to the event, transport it

without confusing records produced by different probes, decode typed arguments, apply user-selected scope, and present the result consistently. When these contracts diverge, valid kernel records may be decoded as unknown events, attributed to the wrong event type, or silently discarded.

Individual observations also have different analytical value. Some are relevant in isolation, for example an access to a sensitive file under narrowly defined conditions. Others become relevant only as an ordered group, such as a credential transition followed shortly by process execution or sensitive file access. Following the established anomaly-detection taxonomy, the thesis uses *point event* for a locally significant observation and *short collective event pattern* for a bounded sequence whose combined meaning is more relevant than its elements considered independently [15]. These terms describe deterministic, rule-based detection; they do not imply machine learning, statistical baselines, or unsupervised anomaly detection methods.

Collecting kernel events alone does not make the resulting telemetry immediately useful for security analysis. The proposed system therefore introduces a configurable policy and detection layer that transforms selected runtime observations into focused security information. Policies define the monitoring scope by selecting the event domain relevant to a particular scenario, helping to limit noise and unnecessary processing. Detectors operate on the resulting normalised event stream and encode reusable conditions that can generate an alert from either an individual event or a short, locally correlated sequence. This separation allows collection scope and detection logic to evolve independently, without requiring each analytical change to be implemented inside the eBPF programs. The generated alerts are enriched with metadata based on MITRE ATT&CK [28], providing a standard vocabulary for interpreting the observed behaviour and supporting downstream analysis.

Four constraints shape the problem. First, the target kernel limits which attachment mechanisms, helpers, maps, and portability techniques can be used reliably. Second, the kernel verifier constrains program complexity and memory access, so event enrichment must be divided carefully between kernel and user space. Third, high-frequency hooks can generate enough data to make transport, decoding, rule evaluation, and output formatting more expensive than the probe itself. Fourth, the agent is expected to run beside production workloads; CPU and memory use are therefore evaluation criteria rather than secondary implementation details.

These constraints motivate a selective architecture. The proposed tool uses

eBPF for collection and optional early filtering, while user space owns event normalisation, policy management, detector evaluation, alert generation, and presentation. The user-space runtime is implemented in Go, which was selected in part because libraries such as `libbpfgo` provide Go bindings to `libbpf` for loading eBPF objects and managing programs, maps, and event buffers [3]. The design preserves a stable kernel-to-user-space contract while allowing detection logic to evolve without recompiling every eBPF program.

1.3 Objectives and Thesis Structure

The main objective of this thesis is to design, implement, and evaluate an eBPF-based runtime security monitoring prototype that collects process and security-relevant kernel events on the target Rocky Linux environment, normalises them in user space, and evaluates them through configurable policies and local detectors.

The remainder of the thesis develops this objective progressively, moving from the necessary technical foundations to the architecture, implementation, and evaluation of the proposed system:

- **Chapter 2 - eBPF Architecture** introduces Linux runtime instrumentation, the eBPF execution model, object loading and attachment, maps, transport, and portability. It also reviews related eBPF-based runtime-security tools.
- **Chapter 3 - Requirements and System Design** defines the target environment, functional and operational requirements, architectural components, event flow, and the boundaries between kernel-space collection and user-space processing.
- **Chapter 4 - Kernel-Space Event Collection Implementation** describes the shared eBPF event framework, process and security hook families, binary record contract, perf-buffer transport, early filtering, and compatibility work required by the target kernel.
- **Chapter 5 - User-Space and Detection Implementation** presents the Go runtime, probe registry, attachment lifecycle, decoder, event selection, policy and detector subsystems, MITRE ATT&CK metadata, output handling, structured logging, and container packaging.

- **Chapter 6 - Experimental Evaluation** evaluates functional behaviour, event and detector operation, CPU and memory consumption, and the effects of the implemented filtering and performance controls on the target Rocky Linux environment.
- **Chapter 7 - Conclusions** summarises the results, discusses the principal limitations of the prototype, and identifies possible directions for future development.

Chapter 2

eBPF Architecture

2.1 Linux Kernel Observability

Runtime monitoring requires evidence about operations while they are being executed. Application logs describe domain-specific actions, but their content depends on what each application chooses to record. The Linux kernel offers a complementary observation layer because it mediates process creation, credential changes, file access, memory management, inter-process communication, and other security-relevant operations. Kernel observability is therefore useful when behaviour crosses application boundaries or is not represented consistently in application logs [46, 18].

Linux does not expose all runtime activity through one universal event source. Instead, it provides instrumentation points at different layers of kernel execution. Each mechanism determines when monitoring code runs, which context is available, and how strongly the observation depends on kernel internals.

System-call observation. A system call is the controlled interface through which a user-space process requests a service from the kernel. It is not itself an instrumentation mechanism. Monitoring systems commonly observe this boundary through system-call tracepoints, such as entry and exit events. An entry event exposes the requested operation and its original arguments, whereas an exit event exposes the returned result. Correlating the two can distinguish an attempted operation from one that completed successfully. This view is broad and process-oriented, but it remains close to the user-space request: one system call may traverse several internal functions, and

its original arguments may not identify the final kernel object or security decision [51, 46].

Tracepoints and raw tracepoints. A tracepoint is a static observation point deliberately inserted into the kernel source by kernel developers. It has a name, belongs to an event category, and defines the data made available when that point is reached. A tracepoint remains inactive until a tracing component is attached to it. In an eBPF-based monitor, this component is an eBPF program loaded from user space and attached to the selected tracepoint. Each time kernel execution reaches that point, the kernel invokes the attached program and provides a tracepoint-specific context containing the declared event fields.

A regular tracepoint gives the eBPF program a structured event context whose layout follows the format exported for that tracepoint. A raw tracepoint refers to the same underlying kernel instrumentation site, but exposes the original arguments passed by the kernel to that site with less intermediate transformation. This reduces preparation performed before the eBPF program runs, while requiring the program to know how each raw argument must be interpreted. Tracepoints are therefore explicit and comparatively stable observation points, whereas raw tracepoints trade some of that abstraction for more direct access to the underlying arguments. Both mechanisms are limited to the tracepoints actually provided by the target kernel [51, 49].

Kprobes and kretprobes. A kprobe is a dynamic probe placed at a selected kernel instruction at run time. Monitoring tools usually identify that instruction through the symbol of a kernel function and place the probe at its entry. Unlike a tracepoint, the observation point does not need to have been prepared explicitly by the kernel developer. When execution reaches the probed instruction, the kernel temporarily transfers control to the probing infrastructure, which invokes the attached eBPF program before normal execution continues. The program receives a snapshot of the processor-register state. Function arguments are not presented as a tracepoint-specific event structure; they must be recovered from the appropriate registers according to the architecture and calling convention.

A kretprobe observes the corresponding function-return path. It causes an attached program to run when an invocation of the probed function finishes and makes the return value available through the register context. A kprobe can therefore describe the operation requested at function entry, while a

kretprobe can reveal the result produced at function exit. Entry and return records are separate observations: when both the original arguments and the result are required in one logical event, the monitoring system must save the entry data and correlate it with the return observation, commonly by using the current thread identity.

Kprobes and kretprobes provide access to functions for which no tracepoint exists, but this flexibility increases coupling to kernel implementation details. A renamed or inlined function, a changed signature, a different calling convention, or an unavailable symbol can invalidate the attachment or the argument-decoding logic. They are therefore powerful dynamic mechanisms, but generally require more target-specific validation than tracepoints [43].

Linux Security Module hooks. The LSM framework inserts security mediation points into kernel operations involving files, credentials, processes, sockets, and other protected objects. At these points, registered security modules can inspect an operation and, for hooks with authorisation semantics, permit or reject it. Observing an LSM-related function can therefore expose a resolved kernel object and the security-relevant stage of an operation, rather than only the original system-call request. Two concepts should not be confused: an LSM hook is the kernel mediation point, while an eBPF LSM program is one possible program type for attaching logic to supported hooks. On kernels where that program type is unavailable or unsuitable, monitoring tools may instead observe the corresponding security function through a kprobe. An entry-side observation still does not necessarily reveal the final outcome; when the result matters, it must be obtained from a compatible return-side observation [45, 49, 43].

Observation source	Information exposed	Main trade-off
System-call tracepoint	Arguments at entry or return value at exit.	Broad process visibility, but limited insight into internal kernel processing and resolved objects.
Kernel tracepoint	Structured fields defined for a specific kernel event.	Clear semantics and simple decoding, but only where a suitable tracepoint is available.
Raw tracepoint	Original tracepoint arguments with minimal preprocessing.	More direct access, but requires kernel-aware argument decoding.
Kprobe / kretprobe	Register state at function entry or return.	Flexible placement, but stronger dependence on kernel symbols, signatures and calling conventions.
LSM-related hook	Security context, resolved objects and mediation stage.	Rich security semantics, but attachment and result visibility depend on target-kernel support.

Table 2.1: Comparison of the principal observation sources relevant to eBPF-based runtime monitoring [51, 43, 45, 49].

The role of eBPF. eBPF is the execution facility attached to an observation point, not a replacement name for the observation point itself. The selected hook causes an eBPF program to run and determines the context received by that program. The program can then extract selected fields, apply an early filter, update maps, or submit an event to user space. Consequently, “tracepoint”, “kprobe”, “LSM hook”, and “eBPF program” describe different parts of the same monitoring path [22, 49].

These mechanisms are complementary. System-call observations describe requests and results at the user–kernel boundary; general tracepoints expose kernel-defined events; kprobes and kretprobes provide dynamic access to function entry and return; and LSM-related hooks expose security mediation points. A runtime monitor combines them according to the required semantics, available target-kernel features, and acceptable coupling to kernel internals. The engineering objective is to select observation points that provide sufficient and reliable evidence for the intended security analysis [46, 21].

2.2 eBPF Fundamentals

eBPF is a modern technology that makes the Linux kernel programmable at runtime. It allows small programs to be loaded from user space and executed inside the kernel when a selected event occurs. Before execution, the kernel verifies that each program respects the safety constraints associated with its program type and intended attachment point [29, 22].

This approach makes it possible to extend kernel behaviour without modifying the kernel source or developing a traditional kernel module for each new functionality. eBPF programs can be loaded, replaced, and specialised for different purposes, including system observability, performance analysis, network processing, and security. Its principal benefit is therefore controlled programmability: functionality can be introduced close to the kernel operations of interest while remaining subject to verification and well-defined execution constraints [18, 23].

2.2.1 Execution Model, Verifier and JIT Compilation

An eBPF program is compiled into instructions for the BPF virtual machine and submitted to the kernel through the BPF user-space interface. The instruction set defines general-purpose registers, a bounded stack, arithmetic and branch instructions, memory operations, and calls to kernel-provided functions. The program does not execute as unrestricted kernel code: its available context, memory accesses, and callable functions depend on its program type [29, 24].

Because the programs described above are loaded from user space but execute inside the kernel, they cannot be allowed to run without first establishing that they respect the kernel’s safety constraints. An unsafe memory access or an invalid operation in this context could affect the stability and security of the entire system. Linux addresses this requirement through the eBPF verifier, the kernel component responsible for deciding whether a submitted program can be accepted for execution. Rather than trusting the program, the verifier analyses it during the loading process and before permitting its execution; a program that does not satisfy the required safety conditions is rejected and never executed.

The verifier performs a path-sensitive static analysis of the program’s control flow. It follows the possible execution paths while tracking the state of registers and stack locations, distinguishing scalar values from different

classes of pointers, checking memory bounds and alignment, and validating the arguments supplied to helper functions. Paths that may access invalid memory, use uninitialised values, or leave kernel-managed resources unreleased are rejected. To avoid analysing equivalent situations repeatedly, the verifier uses state pruning: when a previously accepted verifier state already covers a new state reached at the same instruction, the redundant path does not need to be explored again [23, 19].

Verification is a necessary safety gate, not a proof that a program implements the intended monitoring semantics. A verifier-accepted program may still read the wrong field, emit an incomplete event, or encode data inconsistently with its user-space decoder. Conversely, semantically reasonable source code may be rejected because the verifier cannot establish the required safety properties for a particular kernel. Verifier behaviour is therefore both a security mechanism and a practical compatibility constraint [23, 35].

Once the verifier accepts a program, the kernel may pass its eBPF instructions to an architecture-specific JIT compiler. The compiler translates the verified eBPF bytecode into native machine instructions for the host architecture. The resulting native code can then be executed directly whenever the program is invoked, avoiding the overhead of processing the eBPF instructions individually at each execution [41, 42].

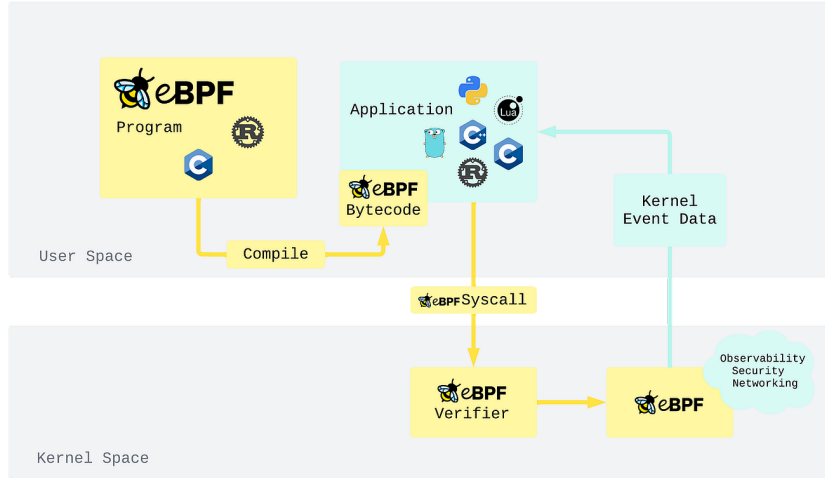


Figure 2.1: High-level eBPF execution workflow.

JIT compilation does not alter the functionality authorised by the verifier.

The translation is performed only after verification and preserves the semantics of the accepted eBPF program. Its availability depends on the host architecture, the kernel build configuration, and the runtime JIT settings.

This completes the basic eBPF execution path considered here. A program is compiled into eBPF instructions, submitted to the kernel, analysed by the verifier and, if accepted, translated into native machine code and executed whenever its associated hook or execution context is invoked [18].

2.2.2 Object Files, Loading and Attachment Lifecycle

The source code of an eBPF application is not passed directly to the kernel. Clang first compiles it into BPF instructions stored in an ELF object. The object groups programs into named sections and also carries map declarations, licence information, symbols, BTF metadata and relocation records. Section names help a loader infer the program type and expected attachment family, while BTF and relocation information describe references that must be adapted before the program can be accepted by the target kernel [49, 40, 36].

A user-space loader, commonly built on `libbpf`, opens this object and turns its declarations into kernel objects. It creates or reuses the required maps, resolves relocations and submits each enabled program through the BPF system-call interface. The kernel then applies the verifier to every submitted program. Loading is therefore an active compatibility boundary: a syntactically valid object may still fail because a required program type, helper, kernel symbol or verifier behaviour is unavailable on the host [25, 44, 23].

Loading and attachment are separate operations. A successfully loaded program exists as a kernel object referenced from user space, but it does not run until it is connected to a compatible hook. Depending on the program and hook type, this connection may be represented by a BPF link or by tracing-specific infrastructure such as a perf event. The attachment owns the relationship between the program and its trigger and can be removed independently from the program itself. This separation allows one object file to contain several programs, only a subset of which may be loaded and attached for a particular monitoring configuration [49, 44].

File descriptors provide the user-space references through which programs, maps and links are configured and eventually released. Closing the final reference normally permits the corresponding kernel object to be destroyed,

whereas pinning can give an object a persistent name in the BPF filesystem and allow it to outlive the process that created it. Runtime ownership and cleanup are consequently part of the operational model, not merely loader housekeeping: stale attachments or pinned objects may continue consuming kernel resources after the original control path has ended [25, 44].

2.2.3 Hook Selection and Event Semantics

An eBPF program type defines the execution contract enforced by the kernel, including the expected context, available helpers, and permitted return semantics. The hook instead identifies the event or kernel execution point that triggers the program, while the attachment mechanism binds the loaded program to a compatible hook. These concepts are related, but they are not interchangeable [49, 24].

The selected observation point determines not only when an eBPF program runs, but also the meaning of the resulting event. A system-call entry hook captures the operation requested by a process and its original arguments. A later tracepoint, dynamic probe, or LSM-related hook may expose kernel-resolved objects or the security mediation stage of the operation. An exit or return-side observation can instead reveal whether the operation completed successfully [51, 43, 45].

These observations represent different stages of the same kernel execution path and should therefore be considered complementary rather than interchangeable. No single hook necessarily exposes the original request, the resolved kernel context, and the final result. A runtime monitor may consequently combine multiple event sources and correlate their records through process identity and temporal proximity.

Hook selection also introduces a compatibility trade-off. Explicit tracepoints provide structured interfaces when the target kernel exports a suitable event, whereas dynamic probes offer broader placement flexibility but depend more strongly on kernel symbols, function signatures, and calling conventions. The appropriate observation point must therefore balance the required security semantics with the capabilities and stability of the deployment kernel [51, 43, 27].

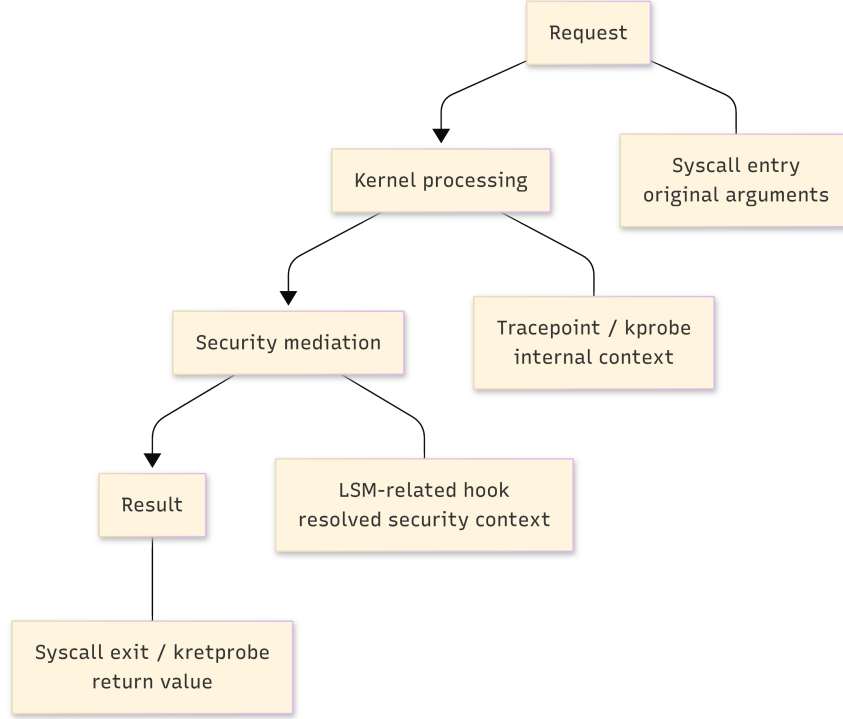


Figure 2.2: Complementary evidence exposed by different observation points along a kernel operation.

2.2.4 Maps, Helpers and Kernel-to-Userspace Transport

The execution of an eBPF program is intentionally constrained and usually short-lived. Runtime monitoring, however, requires information to persist beyond a single invocation and eventually reach user space, where more complex processing can be performed. Linux supports this interaction through kernel-managed state and a controlled communication path between eBPF programs and the user-space runtime. Maps, helpers, and event buffers form the main building blocks of this path [37, 24, 48, 38].

Maps are typed kernel objects that allow eBPF programs to preserve state across invocations and exchange data with user space. Each map type defines its own access and concurrency semantics, while its structure and capacity are established at creation time. By moving persistent configuration and temporary state outside the constrained BPF stack, maps extend the

information that a program can retain between executions. Accesses performed by eBPF code remain subject to verifier validation and to the rules imposed by the selected map type [37, 25].

Different map organisations serve different ownership patterns. Array maps provide fixed indexed storage and are suitable for configuration or bounded workspaces. Hash maps associate dynamically selected keys with values and can retain state such as entry arguments until a later hook completes an operation. Per-CPU variants allocate an independent value for each processor, reducing contention when programs update temporary state or counters on a hot path. Their values must, however, be combined explicitly when user space needs a system-wide result. Shared maps instead require the application to consider concurrent access from several CPUs and the atomicity of each update [37].

Maps also form a control channel in the opposite direction. User space can populate configuration, selection or lookup data before monitoring begins and update supported values while programs are loaded. An eBPF program can then consult that state without embedding every operational choice in its instructions. This division is central to configurable monitoring: the kernel program remains bounded, while user space controls which state it exposes and how map contents are interpreted [37, 25].

Helpers are kernel-provided functions that extend what an eBPF program can do without granting unrestricted access to kernel internals. They expose selected services through a controlled interface, allowing the kernel to validate how those services are invoked and what data is passed to them. The set of available helpers is determined by the program type and by the capabilities of the target kernel, so support documented for eBPF in general does not imply that the same helper can be used in every execution context. This constraint is particularly relevant to this work, which targets Rocky Linux 8.10 with kernel 4.18.0-553.109.1.el8_10.x86_64: helper selection must therefore be validated against the capabilities of the deployed kernel and the program type used by each probe, rather than inferred solely from current upstream documentation [24, 23].

Event transport carries records produced during eBPF execution to user space, where they can be decoded and processed without extending the work performed inside the kernel. The BPF ring buffer provides a shared multi-producer, single-consumer channel designed to use buffer capacity across CPUs and preserve the order in which records are reserved. Perf-event transport follows a different model: a `BPF_MAP_TYPE_PERF_EVENT_ARRAY` directs

records to per-CPU perf ring buffers, which are consumed independently and can report lost events.

Both mechanisms rely on finite buffer capacity. When kernel-side producers generate data faster than user space can consume it, a submission may fail or a record may be lost. Event transport must therefore be treated as part of the monitor’s correctness model rather than as a transparent delivery channel [38, 48, 24].

Mechanism	Buffer organisation	Main implication
BPF ring buffer	Shared multi-producer, single-consumer buffer	Shares capacity across CPUs and preserves reservation order between producers.
Perf-event transport	Per-CPU buffers referenced through a perf-event array	Buffers are consumed independently and provide explicit lost-event notifications.

Table 2.2: Comparison of eBPF event-transport mechanisms [38, 48].

Regardless of the selected transport mechanism, kernel and user space must share a consistent record format so that the transferred data can be decoded into correctly typed events [18].

2.2.5 BTF and CO-RE Portability

An eBPF program often interacts with kernel data structures whose layout may change across kernel versions and distribution builds. A program compiled against one specific set of kernel headers may therefore rely on field offsets or type definitions that do not match the target system. Portability requires a way to preserve the meaning of those accesses while adapting them to the types exposed by the deployed kernel. BTF and CO-RE provide the metadata and relocation mechanisms used to address this problem.

BTF represents types and related metadata in a compact format understood by the kernel and BPF tooling. BTF may describe C types and strings and can also carry function, line, and relocation information. When kernel BTF is available, loaders and debugging tools can reason about kernel data structures without depending solely on a set of headers copied from one kernel build [40].

CO-RE uses BTF information to adapt an eBPF program to the kernel

on which it is loaded. During compilation, Clang stores relocation records that describe the kernel types and fields accessed by the program. At load time, `libbpf` reads the BTF information of the running kernel and updates the relevant eBPF instructions so that they use the actual layout of those structures. As a result, the same compiled object can continue to work when compatible kernel versions place a field at a different offset, without requiring the program to be recompiled on the monitored system [40, 36, 44].

In this thesis, this mechanism helps the eBPF programs access kernel structures according to the layout exposed by Rocky Linux 8.10 with kernel `4.18.0-553.109.1.el8_10.x86_64`. However, CO-RE only adapts compatible differences in type and data-structure layout. It cannot make a missing hook, helper, program type, or attachment mechanism available, and it does not ensure that the target verifier will accept the program. These capabilities must therefore be checked directly on the deployed kernel. This is especially important for enterprise distributions, where backported features may make the effective kernel capabilities differ from those suggested by the upstream version number alone [35, 27].

2.3 Related eBPF Runtime Security Tools

Existing eBPF-based runtime security tools share the objective of interpreting kernel-derived activity, but they place event selection, detection, and response at different points of the monitoring pipeline. Tracee, Falco, and Tetragon are considered here because they represent three relevant design approaches for positioning the proposed system. Tracee provides the closest architectural reference through its separation of event collection, policies, detectors, and output. Falco centres its design on declarative rules and alert generation, while Tetragon moves filtering and optional response closer to the monitored kernel hook [21].

2.3.1 Tracee

Among the systems considered in this section, Tracee is the closest reference for the tool developed in this work. Both systems separate kernel-level event collection from the analysis performed in user space. This shared structure, rather than a direct feature-by-feature correspondence, makes Tracee the main architectural reference for the proposed system [8, 9, 21].

2.3.1.1 Architecture and Event Pipeline

The official architecture overview reproduced in Figure 2.3 presents Tracee as a sequence of connected stages. eBPF programs attached to kernel and networking hooks generate runtime events, which are transferred to user space for collection and enrichment. The resulting observations are then evaluated by the detection layer and made available to external consumers. The diagram therefore illustrates an important architectural property of Tracee: kernel instrumentation is only the first stage of a broader processing pipeline [4].

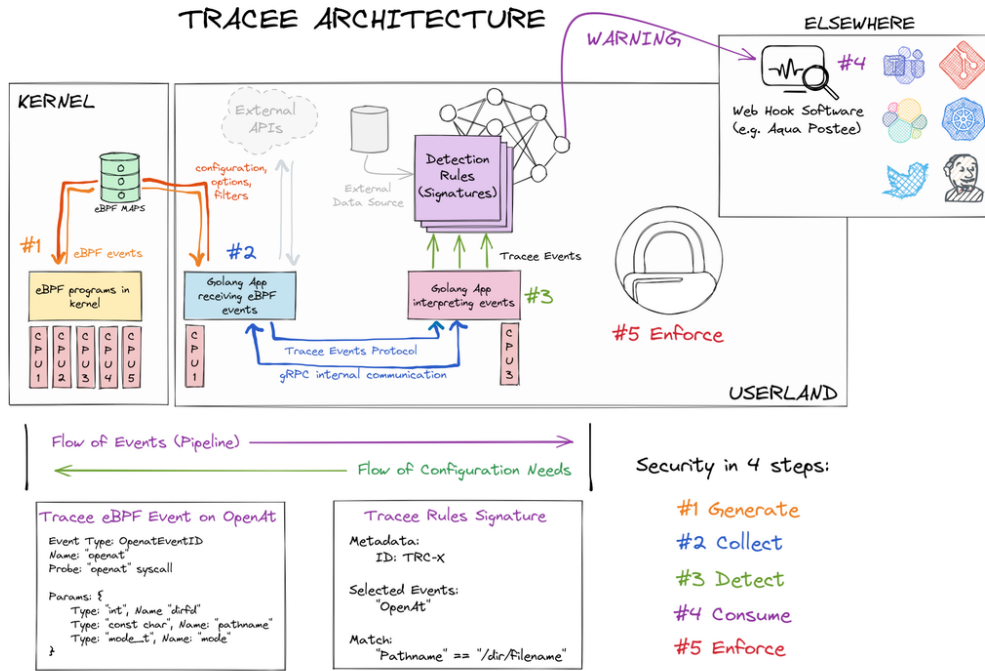


Figure 2.3: Tracee high-level architecture [4].

The terminology used in the figure reflects the Tracee version for which it was published. In particular, the detection stage is represented through the signature-based model used by that release. The current development documentation retains the same broad separation between collection, interpretation, and output, but identifies detectors as the preferred mechanism for

implementing new threat detections and derived events. The figure should therefore be read as a high-level architectural overview. [9, 5, 7].

This evolution does not alter the principal architectural idea illustrated by the diagram. Runtime observations originate from different kernel sources but are converted into events before higher-level processing takes place. Detection logic operates on this event representation rather than interacting directly with each underlying probe, while later components determine how the resulting events are enriched, selected, and delivered.

2.3.1.2 Unified Event Model

The processing stages shown in Figure 2.3 are connected through Tracee’s event abstraction. Observations may originate from system calls, process lifecycle hooks, file and network activity, Linux Security Module instrumentation, or container-related sources, but they are presented to the remainder of the system through a common representation. Policies, detectors, and output components can consequently operate on events without having to manage the attachment semantics of each underlying kernel source [10, 9].

The same abstraction also represents information derived after the original kernel observation has reached user space. Security detections and other derived results become events of their own and can participate in the same processing infrastructure. This reduces the distinction between raw telemetry and higher-level results at the pipeline level, while preserving their different semantic roles [9, 7, 6].

Events can be selected individually or through sets that group related event types. The event catalogue therefore acts both as the interface exposed to users and as the vocabulary shared by the principal user-space components [10].

2.3.1.3 Policies and Event Selection

Tracee policies determine which part of the available event domain is relevant to a monitoring scenario. A policy combines a scope, which identifies the workloads or execution contexts to which it applies, with rules that select events and optionally constrain their data. The same policy model can be expressed through a plain YAML or JSON document or through a Kubernetes custom resource, allowing the monitoring intent to remain consistent across local and orchestrated deployments [12].

This abstraction separates the decision to observe an event from the logic that interprets its security significance. Policies can restrict collection to the processes, containers, or event families of interest without embedding every selection rule inside a detector. Where the necessary information is available during kernel-side processing, filtering can prevent irrelevant records from entering the user-space pipeline. Conditions that require richer context can instead be completed after the event has been collected and normalised [12].

In the current development model, policy rules can refer not only to ordinary runtime events but also to detectors through their threat metadata. A policy may therefore select detections according to properties such as severity or MITRE ATT&CK classification without naming every detector individually. This selection is resolved when Tracee starts, so detectors that do not satisfy the configured criteria do not need to participate in runtime dispatch [13].

The policy layer consequently provides more than a final output filter. It defines the portion of the event and detector domain that Tracee prepares for a particular monitoring objective, while leaving the interpretation of matching observations to the detector layer.

2.3.1.4 Detectors and Derived Events

The signature block shown in Figure 2.3 represents the detection model used when that architecture was documented. Tracee’s current development version introduces detectors as the preferred mechanism for new threat detections and derived events, while retaining the earlier signature system for backward compatibility [5].

A detector declares the input events required by its logic and receives matching observations from the event pipeline. It can then evaluate those observations and produce a derived event when its condition is satisfied. Detectors may be implemented in Go when programmatic control and access to runtime state are required, or described through YAML when the desired condition can be expressed declaratively [7, 6].

Because detector results are represented as events, they are not restricted to a terminal notification channel. They can be selected by policies, routed through the output infrastructure, or consumed by other detectors. Provenance information can preserve the relationship between a derived result and the observations from which it originated [6].

The detector API permits implementations to consume several event types

and retain internal state. This makes stateful detection possible, but it should not be interpreted as a predefined general-purpose temporal language. The API provides the mechanisms through which a detector may combine evidence, while the meaning and limits of that correlation remain part of the detector implementation.

2.3.1.5 Output and Operational Context

Once events have passed through collection, selection, and detection, Tracee's output subsystem determines how they are presented and delivered. Output configuration is independent from the detector that produced an event. The same observation can therefore be rendered for interactive inspection or serialised for machine processing without changing its detection logic [11].

Tracee supports human-readable tables, structured JSON records, and custom templates. Events can be written to local destinations, forwarded to external logging infrastructure, or delivered through webhooks. Output streams provide an additional routing layer by allowing different portions of the event domain to be sent to independent destinations [11].

The event stream may also be enriched with information maintained outside the original kernel record. Process ancestry and container context can provide the operator with a clearer account of the workload responsible for an observation, while provenance information connects a derived detection to the event from which it originated. This enrichment illustrates why Tracee keeps event collection distinct from the later interpretation and presentation stages: the eBPF program captures the information available at the hook, while user space can relate that record to broader runtime state [6, 11].

Tracee additionally includes forensic capabilities that extend beyond alert generation. Runtime events can guide the collection of artefacts for later investigation, giving the system a broader operational scope than a monitor whose output is limited to telemetry and detector alerts [9].

2.3.1.6 Portability and Security Boundaries

Tracee's breadth depends on capabilities exposed by the monitored Linux kernel. Its current installation guidance identifies BTF information and kernel-symbol availability as important prerequisites and explicitly recognises the enterprise Linux 4.18 kernel used by Red Hat Enterprise Linux

8 as a supported exception to its general upstream-version guidance. Individual events may nevertheless depend on additional kernel configuration or attachment support, so the presence of the minimum kernel version does not imply that every Tracee capability is available [2].

This compatibility model is particularly relevant to the target considered in this thesis. It demonstrates that a long-lived enterprise kernel cannot be evaluated solely through its nominal upstream version. BTF availability, vendor backports, accessible symbols, verifier behaviour, and supported hooks must be examined on the deployed system.

Tracee also states an explicit trust assumption: the kernel is expected to be trustworthy when monitoring begins. Once a kernel-level adversary controls the system, it may interfere with eBPF execution, suppress observations, or disable the monitoring infrastructure. Tracee can attempt to identify known forms of kernel manipulation after startup, but such detection remains best effort rather than a guarantee against a compromised kernel [14].

This boundary is important when interpreting kernel-derived telemetry. Running the collection programs inside the kernel protects them from ordinary application behaviour, but it does not make the monitoring system independent from the integrity of the kernel on which those programs execute.

2.3.1.7 Relationship to the Proposed Work

Tracee influenced the proposed system primarily through its separation of event collection, policy-based selection, detector evaluation, and output. The tool developed in this work adopts the same general division of responsibilities: heterogeneous kernel observations are converted into a normalised user-space representation, policies define the monitored event domain, and detectors interpret selected observations without placing all security logic inside the eBPF programs.

The relationship is architectural rather than one of functional equivalence. Tracee provides a broad event catalogue, extensive process and workload context, configurable routing, detector chaining, and forensic capabilities. The prototype developed in this thesis instead concentrates on a selected set of process and host-security events on the specified Rocky Linux kernel. Its detector model is deliberately bounded around deterministic conditions over individual events and short locally correlated sequences.

Tracee therefore serves both as a design reference and as a point of compar-

ison for the contribution of this work. It shows that the main architectural choices on which the proposed system relies are already established in runtime security tooling. The contribution of the thesis lies instead in bringing these ideas together within an independently developed prototype tailored to the selected enterprise environment, where its compatibility, internal event contract, collective detection model, and runtime behaviour can be examined directly.

2.3.2 Other Relevant eBPF Security Tools

Falco is a runtime security engine organised around rules evaluated over event sources. Kernel system-call activity can be captured through its driver architecture, while plugins provide additional sources. Falco rules are declarative YAML objects containing a condition, output description, priority, and optional reusable lists, macros, tags, and exceptions. A matching rule produces an alert that can be routed to configured outputs [31, 34, 32].

This model demonstrates the operational value of readable rules, reusable expressions, explicit exceptions, and an alert-oriented interface. Falco also maintains process and container context used to enrich individual events. However, its documented rule conditions are evaluated against an incoming event, and separate event sources are processed independently; the core rule language is not presented as a general temporal sequence language across heterogeneous sources [30, 33].

Tetragon combines eBPF-based observability with optional runtime enforcement. Its tracing policies specify kernel hooks, captured arguments or return values, and selectors that can match process, namespace, capability, workload, and argument attributes. Supported actions can include emitting an event, delivering a signal, or overriding a return value where the hook and kernel support that operation [55, 56, 52].

Tetragon therefore moves more selection and potential response close to the hook than an architecture centred on user-space detectors. It preserves process-lifecycle context and emits structured events through its own interfaces, but its documented tracing-policy model is principally hook-centred rather than a general language for arbitrary temporal sequences [53, 55]. Its current installation requirements also illustrate the dependence of advanced eBPF functionality on kernel capabilities and BTF availability [54].

2.3.3 Comparative Summary

The systems examined in this section represent different approaches to runtime security. Tracee provides the closest architectural reference because it separates kernel-level collection from the analysis performed in user space. Falco instead places declarative rule evaluation and alert generation at the centre of its design, while Tetragon moves a greater part of event selection and optional response closer to the kernel hook.

These approaches are not directly interchangeable, but they provide useful reference points for the proposed system. Tracee illustrates the event-oriented pipeline that most closely influenced its architecture, whereas Falco and Tetragon establish the boundaries represented by an alert-centred rule engine and an enforcement-oriented design. The main distinctions are summarised in Table 2.3.

Tool	Core approach	Role in the comparison
Tracee	Event-oriented pipeline with separate policy, detection, and output stages.	Closest architectural reference for the proposed system. [12, 7]
Falco	Declarative rules evaluated over incoming events to generate alerts.	Reference for a mature rule- and alert-centred model. [34, 33]
Tetragon	Hook-centred policies with in-kernel selectors and optional actions.	Reference for kernel-side filtering and runtime enforcement. [55, 52]

Table 2.3: Architectural summary of the runtime security tools considered in this chapter.

2.4 Positioning of the Proposed Work

The proposed work occupies a deliberately bounded part of the Linux runtime security design space. It focuses on process and host-security monitoring on Rocky Linux 8.10 with kernel 4.18.0-553.109.1.el8_10.x86_64. Because this environment combines an older upstream kernel base with vendor backports, compatibility cannot be inferred from the version number alone. The available hooks, helpers, data layouts, and verifier behaviour must instead be established on the deployed system [27].

Within this environment, eBPF programs collect selected kernel observations and transfer them to a Go runtime through an explicit event contract. The resulting records become a typed user-space event stream. Chapter 3 defines how that stream is scoped through policies and interpreted by detectors; the present chapter is concerned with the eBPF collection model and the prior-art systems against which the complete architecture is positioned.

The individual mechanisms used by the prototype have clear precedents in existing runtime security systems. Tracee provides the closest architectural reference through its event-oriented separation of collection, policy selection, detection, and output. Falco demonstrates the maturity of declarative event rules and alert generation, while Tetragon illustrates a more kernel-centred design that also supports enforcement [12, 7, 34, 55].

The contribution of this thesis therefore lies in the independent integration of these ideas within a compact prototype tailored to the selected enterprise kernel. Particular attention is given to the compatibility decisions required by that environment, the selection of complementary observation points and the contract through which kernel records become typed user-space events. The policy and detection architecture built on that contract is developed in Chapter 3 rather than repeated here.

Chapter 3

Requirements and System Design

The previous chapter established the technical basis for the work and positioned the proposed system within the runtime security landscape. Building on that foundation, this chapter defines the requirements that guide the system and develops the architecture chosen to satisfy them. The discussion focuses on the constraints imposed by the target environment and on the division of responsibility between kernel space and user space.

The design is guided by a simple principle: work that depends directly on the observed kernel operation should remain small and bounded, whereas higher-level interpretation should take place in user space, where it can evolve without increasing verifier-sensitive kernel logic. An explicit event contract connects the two domains. Once an observation crosses this boundary, policies determine whether it belongs to the monitored scope, while independently configured detectors evaluate individual events or short ordered sequences. These choices are presented here at the architectural level. Their kernel-space implementation is developed in Chapter 4, the user-space runtime and detection implementation in Chapter 5, and their experimental assessment in Chapter 6.

3.1 Design Goals and Constraints

The system is designed to provide focused visibility into security-relevant host activity rather than an exhaustive account of kernel execution. Its event domain therefore centres on process execution and on operations that affect the security state of the monitored host. The architecture must allow this domain to be selected according to the monitoring objective, while presenting the resulting observations through a consistent typed event model. Security interpretation should then operate on that model without requiring the detection logic to be embedded in the eBPF programs.

This design is shaped primarily by the characteristics of the target environment and by the continuous nature of runtime telemetry. Kernel capabilities must be established on the concrete enterprise system, since its reported upstream version does not fully describe the available functionality. At the same time, frequent observations require the system to avoid unnecessary collection and to limit the state retained for ongoing detections. Any loss or failure along the event pipeline must also remain observable, so that missing telemetry is not interpreted as evidence that no relevant activity occurred.

3.1.1 Target Environment and Compatibility Assumptions

The target environment is Rocky Linux 8.10 on x86-64, running the exact kernel build `4.18.0-553.109.1.el8_10.x86_64`. This target belongs to a long-lived enterprise kernel line in which selected fixes and features are backported while the reported upstream base version remains stable. The nominal version therefore cannot be used alone to infer the effective eBPF capabilities of the host; the functionality required by the monitoring system must be confirmed on the deployed kernel [27].

The architecture assumes that the monitored host exposes usable kernel BTF information and grants the privileges required to load and attach the selected programs. As discussed in Chapter 2, BTF and CO-RE can adapt compatible differences in kernel data-structure layout, but they cannot provide missing kernel functionality, bypass permissions, or guarantee verifier acceptance [40, 36, 44, 23].

This target-aware design deliberately bounds the portability claim. The system is designed around the identified Rocky Linux environment and is not assumed to operate unchanged across arbitrary distributions or kernel configurations. Dynamic probes introduce an additional dependence on the

availability of kernel symbols and compatible function signatures. Loading and attachment failures must consequently be detected before monitoring begins, rather than allowing the runtime to continue with an incomplete observation domain.

3.1.2 Functional Requirements

Table 3.1 defines the functional requirements that guide the system architecture. They describe system-level capabilities rather than individual commands or source-code components.

ID	Functional requirement
FR-01	Collect a selectable set of process and security-relevant events from the monitored host.
FR-02	Activate only the kernel observation components required to produce the selected logical events and their dependencies.
FR-03	Transfer kernel records to user space through the selected perf-event transport and convert valid records into normalised typed events according to an explicit shared contract.
FR-04	Load configurable policies that constrain the monitored event and process scope and may restrict the configured detector catalogue before runtime construction.
FR-05	Construct a separate detector engine from the definitions admitted by the active configuration and policies, and evaluate deterministic conditions over single events or bounded ordered sequences.
FR-06	Generate alerts that preserve the observations responsible for a match and propagate the available MITRE ATT&CK metadata.
FR-07	Maintain independent presentation paths for raw events, detector alerts, and operational diagnostics.
FR-08	Provide a controlled runtime lifecycle that validates configuration, reports observable failures, responds to cancellation, and releases acquired resources during shutdown.

Table 3.1: Functional requirements of the proposed system.

Together, these requirements establish the boundaries developed in the remainder of the chapter. FR-01–FR-03 concern collection and the event contract, FR-04–FR-06 define the analytical path, and FR-07–FR-08 govern presentation and runtime lifecycle.

3.1.3 Non-Functional and Operational Requirements

The requirements in Table 3.2 define the compatibility, reliability, and resource constraints under which the functional pipeline must operate.

ID	Non-functional or operational requirement
NFR-01	Operate on the identified enterprise-kernel target with explicit compatibility assumptions and validation against the deployed system.
NFR-02	Keep kernel-side execution and data extraction bounded and acceptable to the target verifier.
NFR-03	Maintain a consistent kernel-to-user-space event contract and visibly reject unknown or malformed records before further processing.
NFR-04	Keep detection deterministic, configuration-driven, and independent from event presentation.
NFR-05	Bound incomplete collective-detection state in both retention time and capacity.
NFR-06	Report setup failures, transport loss, decoding errors, and detector failures through dedicated diagnostic channels, separate from monitored events and generated alerts.
NFR-07	Assess whether steady-state user-space processing remains below 5% of one CPU core for explicitly defined representative profiles.

Table 3.2: Non-functional and operational requirements.

NFR-07 defines an evaluation objective rather than an unconditional performance guarantee. Its assessment therefore depends on the monitored workload, enabled events, selected detectors, output configuration, and measurement method. Chapter 6 defines the representative profiles and procedures used for this evaluation.

3.2 System Architecture

The architecture separates a control path from a data path. The control path prepares the runtime by validating the configuration and rule definitions, determining which events and detectors are admitted, and resolving each logical event to the kernel programs required for its observation. The data path starts when one of these programs is triggered and carries the resulting observation through transport and user-space processing until it is presented as an event or, when a detector condition is satisfied, as an alert. Operational

diagnostics accompany both paths but remain separate from the monitoring data they describe.

3.2.1 Kernel-Space and User-Space Responsibilities

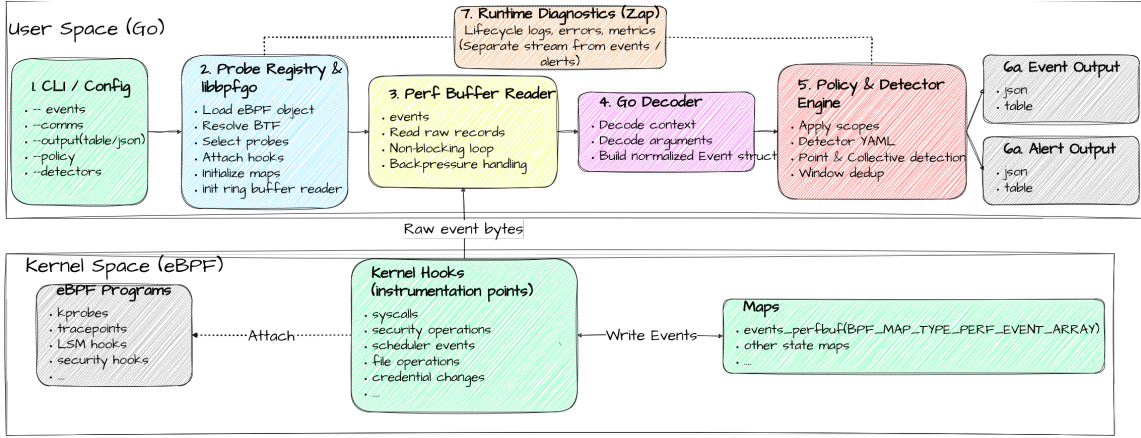


Figure 3.1: Kernel-space and user-space responsibilities.

Figure 3.1 summarises the division of responsibilities between kernel space and user space, together with the control, monitoring, and diagnostic interactions that cross this boundary.

Kernel space is responsible for observing the operations selected during bootstrap and for constructing the corresponding logical events. When an attached hook is reached, the associated eBPF program extracts bounded process context and event-specific arguments, applies the optional exact-match effective-UID guard, and serialises the resulting record according to the shared binary contract. The completed record is then submitted through the perf buffer used as the primary kernel-to-user-space transport channel [24].

Some operations require information that is not available at a single instrumentation point. In these cases, an entry-side program preserves a limited amount of temporary state and a later return-side program completes the event with the operation outcome. This entry/exit model is used only for event construction. It does not introduce policy evaluation or detection logic into the kernel.

User space owns the more changeable and semantically rich responsibilities. During bootstrap, the Go runtime validates the configuration, policies, and detector definitions, derives the effective event scope, resolves the dependencies of the selected logical events, and determines which eBPF programs must be loaded and attached. It manages this lifecycle through `libbpf` and its Go bindings, `libbpfgo` [3]. In this role, `libbpf` provides the user-space facilities required to load eBPF objects and manage programs, maps, and attachments, while `libbpfgo` exposes those facilities to the Go runtime.

During monitoring, the runtime receives binary records from the perf buffer, decodes their context and indexed arguments, and converts them into normalised events. The policy predicates that could not be resolved during bootstrap, including constraints that depend on decoded process information and suppression rules, are then evaluated in user space. Events admitted by this gate may be routed to the event output and are also dispatched to the detector engine constructed from the detector definitions selected during bootstrap. Point detectors evaluate individual events, whereas collective detectors correlate short ordered sequences within bounded temporal and state limits. Only successful detector evaluations produce alerts, which are routed through an output path distinct from raw event presentation.

The diagnostic path is implemented through the structured Zap logging library [57], forming a third independent stream. These diagnostics describe conditions affecting the control or data path while remaining distinct from both monitored events and detector alerts.

The boundary is therefore selective rather than absolute. User space controls which kernel programs are loaded and may configure the narrow kernel-side UID guard, while kernel programs may retain short-lived state needed to complete a record. Higher-level policy interpretation, event normalisation, detection, and presentation remain in user space. Conversely, user-space processing depends on the correctness and delivery of the records constructed in kernel space and cannot recover an observation that was rejected before submission or lost during transport.

3.2.2 End-to-End Event Lifecycle

Before collection begins, the configuration, policy rules, and detector definitions are validated. Where the active policies provide exact event allowlists, these may narrow the event scope requested through the remaining configuration. Policies may also restrict the available detector catalogue before

the detector runtime is constructed. The probe registry then resolves the resulting public logical events into the producer programs and internal dependencies required to construct them, while unrelated registered programs are excluded from loading and attachment.

Once monitoring is active, reaching a selected hook invokes its associated eBPF program. The Go runtime decodes the record according to the event registry and converts it into a normalised event. It then evaluates the remaining event and process constraints whose application depends on decoded user-space context. An admitted event may be routed independently to event presentation and to the relevant detectors selected during bootstrap. Disabling event presentation therefore does not prevent detector evaluation. A successful single-event or collective detector evaluation produces an alert that preserves the source observations responsible for the match and propagates the available MITRE ATT&CK metadata.

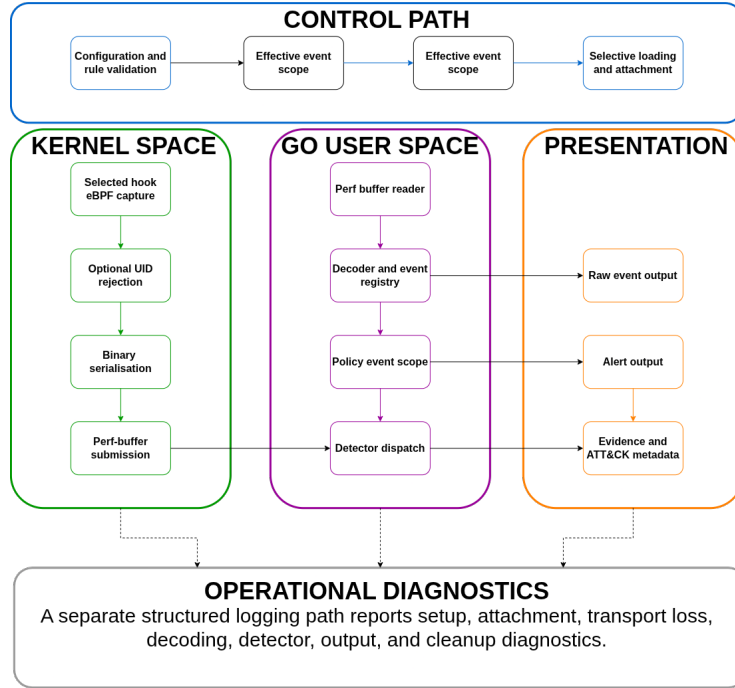


Figure 3.2: Responsibility boundary and end-to-end flow of the proposed monitoring system.

Figure 3.2 summarises the relationship between the control path and the

monitoring data path. Operational diagnostics remain separate because they describe conditions affecting these paths rather than representing monitored events or detector alerts.

The diagram does not imply a total ordering of monitored events. Per-event transport uses per-CPU buffering, and collective detectors therefore interpret sequence order according to the order in which records reach the user-space engine. The corresponding transport-loss and ordering boundaries are discussed further in Section 3.4.2.

3.3 Kernel-Space Collection Design

Kernel instrumentation must expose evidence that is both meaningful for the logical event and obtainable on the target kernel. The design therefore does not prescribe one universal attachment mechanism. It combines available observation points while maintaining a stable event abstraction above them.

3.3.1 Hook Selection and Logical Events

Hook selection begins with the semantics required from the observation. The chosen point must expose the relevant stage of the operation, which may be the initial request, a resolved kernel object, a state transition, or a security mediation decision. An entry-side observation is sufficient when the attempted operation is itself meaningful. When the distinction between success and failure affects the interpretation of the event, a compatible return-side observation is also required.

The choice is further constrained by the observation points available on the concrete target and by their expected cost and robustness. An explicit tracepoint is preferable when it exposes the required semantics, whereas a dynamic probe may extend coverage where no suitable tracepoint exists, at the cost of stronger dependence on kernel symbols, function signatures, and calling conventions. Similarly, a dedicated observation point is preferred over a high-frequency generic hook when it provides equivalent evidence more selectively. These are design-time decisions derived from the properties of the kernel mechanisms discussed in Chapter 2, rather than choices made automatically while the system is running.

Table 3.3 distinguishes the architectural concepts used to describe instrumentation and event production.

Concept	Architectural meaning
Physical hook	A specific point in kernel execution, such as a tracepoint or kernel function, at which an eBPF program can be attached and invoked.
Producer probe	An eBPF program that directly captures an observation and submits a record for a public logical event. A logical event may be produced by one or more producer probes.
Support probe	An internal eBPF program that does not produce a public event of its own. Instead, it stores, derives, or supplies information required by a producer probe to construct another event correctly.
Logical event	The stable user-space event type exposed by the monitoring system, identified by a public name and decoding schema. It hides the number and type of physical hooks and probes used to construct the observation.

Table 3.3: Distinction between physical hooks, producer and support probes, and logical events.

The relationships between these concepts are not necessarily one-to-one. A logical event may require an entry probe to preserve request information and a return probe to complete the record with the operation outcome. Different operation variants may also converge on the same logical event. Conversely, selecting one public event may activate support probes that maintain required context but do not produce independently decodable records. Separating these attachment units from the public event abstraction prevents target-specific probe mechanics from becoming part of the user-facing event contract.

3.3.2 Early Event Selection and Kernel-Side Filtering

The public event abstraction introduced in the previous subsection also defines the unit of early selection. Rather than requiring the operator to identify individual eBPF programs, the architecture starts from the logical events that the monitoring scenario is expected to produce. For each selected event, the probe registry determines the producer probes responsible for its construction and any support probes on which those producers depend.

The resulting dependency set determines which registered programs enter

the loading lifecycle. Programs that cannot contribute to the selected event domain are excluded before they are submitted to the kernel for verification and loading, and are consequently not attached to their hooks. This approach reduces the active observation surface while preserving the internal probes needed to construct each requested event. It also keeps the public selection interface independent from the particular arrangement of hooks and programs used on the target kernel.

Selective loading applies only to programs represented by the probe registry. It is not a general removal of every unused section contained in the compiled eBPF object, and its correctness depends on the registry describing producers and support dependencies completely. An omitted dependency could leave a selected event without information required for its construction, whereas an incorrectly exposed support probe could make an internal mechanism appear as an independently selectable event.

A further reduction can be applied after an attached program is invoked. The optional exact-UID guard compares the effective UID of the current task with one configured value and rejects non-matching observations before complete context enrichment, record serialisation, and transport. The guard therefore avoids part of the kernel-side and transport work associated with observations known to be outside that narrow scope.

The guard is intentionally limited to a single equality condition and serves as a representative form of inexpensive kernel-side pre-filtering. Additional predicates over values such as process or local cgroup identity could be introduced where they are available reliably and provide a measurable reduction in unnecessary collection. More expressive conditions remain in user space, where they can be evaluated over the complete normalised event without increasing verifier-sensitive kernel logic.

3.4 Event Contract and Transport Design

Kernel-side producers and the user-space consumer require a common representation that remains consistent across heterogeneous event sources. The design combines a bounded binary transport contract with an explicit registry for semantic interpretation. These elements must evolve together: omitting repeated schema metadata from each record reduces transport redundancy, but also means that identifiers, encodings, and registry definitions must remain aligned.

3.4.1 Stable Binary Event Contract

Each public record contains a common event context followed by indexed, typed arguments. The context carries the identity and provenance information shared across event families, including process, host, namespace, and execution context. Event-specific arguments describe the observed operation, while their indices associate the encoded values with the schema maintained by the user-space event registry.

The representation is intentionally not fully self-describing. The kernel producer emits an event identifier and the encoded argument values, whereas the event registry supplies the corresponding public name, argument types, and expected ordering. This avoids transporting the same semantic meta-data with every observation, but creates an explicit compatibility obligation between the kernel definitions, the Go protocol representation, the decoder, and the event registry.

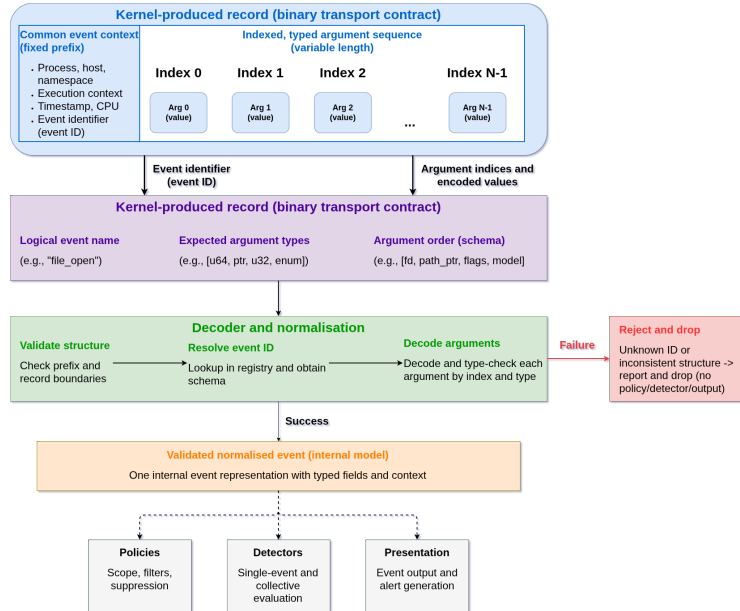


Figure 3.3: Conceptual relationship between the binary event record, the user-space event registry, and the resulting normalised event.

The contract remains valid only while the producer and consumer agree on

the common context layout and each public event identifier resolves to a single compatible schema. The indices, types, and encodings of the supplied arguments must also agree with that schema. A record with an unknown identifier or an inconsistent structure is therefore reported and discarded before it reaches policy evaluation, detector processing, or presentation. The runtime does not convert such a record into a partially trusted event.

Decoding and normalisation are related but distinct stages. The decoder first validates the common record structure and resolves the event identifier through the registry. It can then interpret each indexed argument according to the registered schema and verify that the delivered payload is structurally consistent. Only a successfully decoded record is converted into the common normalised representation consumed by policies, detectors, and output components.

The probe registry does not define the binary schema, but it must remain consistent with the same logical interface. Every probe that produces a public event must refer to an event identifier for which the user-space registry provides a valid decoding contract. Chapter 4 describes the producer and wire contract, while Chapter 5 explains the probe and event registries that preserve these relationships in user space.

3.4.2 Perf-Buffer Transport and Failure Boundaries

The operational event path uses perf-event transport, in which records are delivered through per-CPU perf ring buffers (`BPF_MAP_TYPE_PERF_EVENT_ARRAY`). This mechanism should not be confused with the BPF ring-buffer facility represented by `BPF_MAP_TYPE_RINGBUF`. Although both use circular buffering, the former maintains separate buffers for individual CPUs, whereas the latter provides a shared multi-producer ring buffer [48, 39].

Compatibility with the target kernel was the principal reason for selecting perf-event transport. The perf-event interface is available on the deployed Rocky Linux environment, whereas the BPF ring buffer belongs to a later upstream kernel generation and cannot be assumed to be present through a vendor backport. The transport used by every event was therefore based on the interface verified on the concrete target.

The BPF ring buffer offers advantages on compatible kernels, including shared capacity across CPUs and stronger ordering between records submitted to the same buffer. Perf-event transport instead uses per-CPU buffers,

which may use memory less efficiently under uneven load and do not provide a total ordering across CPUs. These properties were accepted in favour of target compatibility and a simpler operational design.

The choice was also influenced by the evolution of the prototype. An intermediate version maintained both perf-buffer and BPF ring-buffer readers, even though their records eventually entered the same decoding pipeline. Converging on perf-event transport removed duplicated lifecycle and failure-handling paths and ensured that the transport no longer depended on which probe produced the event. This decision does not demonstrate that perf buffers are generally faster, since no controlled comparison between the two mechanisms was performed.

Perf-event delivery remains asynchronous and finite. Records may be lost when a per-CPU buffer cannot be consumed quickly enough, and the resulting loss notification does not allow the missing observations to be reconstructed. A delivered record may instead fail during decoding if it violates the active binary contract. Transport loss and decoding failure are therefore separate conditions, neither of which implies that the corresponding kernel operation did not occur.

The per-CPU organisation also limits ordering guarantees. Collective detectors evaluate sequences according to the order in which records reach the user-space engine, since the architecture does not reconstruct a total causal order across CPUs. The effects of transport pressure, loss, and concurrent delivery are assessed in Chapter 6.

3.5 User-Space Runtime Design

The Go runtime coordinates kernel-facing resources with the higher-level event pipeline. Its components form a controlled runtime lifecycle in which resources are acquired only after validation and remain under clear ownership, while failures affecting individual events are contained without disrupting the main processing loop.

3.5.1 Loading, Attachment and Runtime Lifecycle

The runtime follows an ordered lifecycle that separates setup, active event processing, and cleanup. Figure 3.4 summarises this lifecycle and distinguishes fatal setup failures from event-local failures that do not interrupt

monitoring.

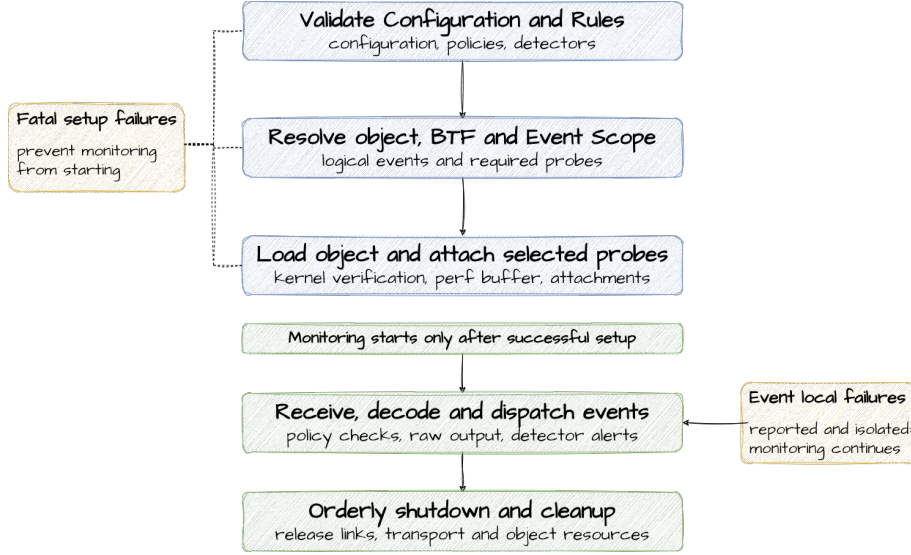


Figure 3.4: Simplified user-space runtime lifecycle.

Startup is ordered so that the configuration and external definitions are accepted before kernel resources are acquired. The runtime normalises and validates the configuration, resolves the eBPF object and BTF source, and then loads and validates the policy and detector definitions. Active policies may narrow the effective event scope and detector catalogue during this bootstrap phase, before the detector runtime is constructed. The selected logical events are subsequently resolved through the probe registry into their producer and support probes, while unrelated registered programs are excluded from autoload.

3.5.2 Decoding, Normalisation and Event Registration

The decoder forms the user-space validation boundary for records received from the perf buffer. It first verifies that the common event context and each encountered argument payload can be read within the delivered record. The event identifier is then resolved through the event registry, and argument indexes are interpreted according to the registered schema. This process produces a typed `bufferdecoder.Event` containing the common context,

the logical event name, and the decoded arguments.

Only a record with a decodable context, a known event identifier, and argument records that can be interpreted using the registered schema enters the policy and detector pipeline. The decoder rejects records that cannot be safely reconciled with the registered schema and configured bounds, for example when the payload is truncated, the event identifier is unknown, or an encoded string exceeds the permitted size. It does not, however, require every schema argument to be present, reject duplicate argument indexes, or require the complete payload to be consumed. The decoding contract therefore establishes bounded and typed interpretation rather than strict canonical encoding.

The event registry and the probe registry describe separate aspects of the same logical interface. The event registry is authoritative for public event identifiers, names, and argument schemas used by decoding and configuration validation. The probe registry instead defines the attachable eBPF programs, their kernel hooks and attachment methods, and any support dependencies needed to produce a logical event.

The separation between the two registries preserves distinct architectural responsibilities: the event registry defines the decoding contract, whereas the probe registry defines how the corresponding observations are produced. Their alignment is enforced through consistency checks that verify that public logical events have a valid decoder definition and that internal support probes remain outside the public event-selection interface. The relationships between physical probes and logical events are already discussed in Section 3.3.1.

The decoded event provides the typed representation passed to subsequent user-space stages, while its use by policies, detectors, and presentation components is discussed in Sections 3.6 and 3.5.3. Records that cannot be decoded are instead reported and discarded at this boundary, preventing unknown or partially interpreted data from entering the normalised event stream while allowing later valid records to continue through the monitoring loop.

3.5.3 Event Output, Alert Output and Operational Logging

The output architecture preserves three semantically distinct result classes: monitored events, detector alerts, and operational diagnostics. This separa-

tion is not merely presentational. Event output exposes telemetry admitted by the active policy scope, alert output exposes security results derived from detector matches, and operational logging describes the state of the monitoring infrastructure itself.

Each class follows an independent output contract. Event presentation is a terminal consumer of the normalised event stream and may be disabled without removing the corresponding observations from policy evaluation or detector dispatch. Alert presentation is similarly separated from detector evaluation: changing its representation does not alter the matching condition or the structured evidence retained by the alert. Presentation decisions therefore do not affect collection and detection semantics.

Operational diagnostics remain outside both security-data paths. The runtime uses the Uber Zap library to provide structured logging for lifecycle state, libbpf messages, transport loss, decoding and detector failures, presentation errors, and cleanup conditions. Zap is used only for diagnostic reporting; it does not serialise monitored events or detector alerts. Events and alerts are handled by their respective output components and retain formats suited to interactive or machine-oriented consumption [57].

This separation prevents infrastructure conditions from being interpreted as monitored behaviour. For example, a decoding failure is reported through Zap rather than being converted into an event, while a detector failure is not represented as a security alert. Diagnostic verbosity and encoding can therefore be adapted to the deployment environment without changing the event or alert interfaces exposed to downstream consumers.

The three paths also define separate failure boundaries. A presentation or diagnostic failure must not invalidate a successfully decoded event, alter detector evaluation, or prevent the remaining paths from progressing. Some startup failures occur before the Zap logger is constructed and are returned directly to the process entry point. This is an outer lifecycle boundary rather than an additional output class; once the managed runtime has been created, Zap provides the structured diagnostic path shown in Figures 3.1 and 3.3.

3.6 Policy and Detection Architecture

Section 3.2 introduced the control and data paths of the complete monitoring system. This section narrows that view to the user-space analytical

subsystem. Its architecture is defined by two moments: bootstrap compiles policy and detector documents into an execution plan, while event-time processing applies that plan to admitted observations. Policies participate in both moments by shaping the plan and retaining the runtime admission rules that could not be resolved earlier. Detectors enter only after catalogue selection and interpret the admitted stream through a shared, indexed engine. This arrangement avoids reparsing configuration or searching the complete detector catalogue on the event hot path.

3.6.1 Policy-Based Event Scope

Policies contribute to the execution plan at two different architectural boundaries. First, their event selectors may reduce collection before the eBPF object is loaded. This optimisation is possible only when every active non-suppressive rule describes an explicit event set. The combined policy set is then intersected with any event selection supplied by the operator, after which the probe registry resolves the producer probes and their internal dependencies. A policy containing an open-ended selector cannot be converted into an exact attachment plan, so the runtime conservatively retains the wider collection scope.

Second, policies reduce the detector catalogue before detector instances are created. The complete catalogue is parsed and validated first; policy selectors then choose definitions by identity or by descriptive properties, including severity, tags, statefulness, and MITRE ATT&CK tactics or techniques. Selection is therefore a configuration-plane operation rather than a condition checked for every event. Excluded definitions allocate no detector or correlation state. The bootstrap validation also checks that a policy selecting a detector admits all events required by that detector, preventing an apparently active analysis from being constructed without its inputs.

The runtime policy manager retains the constraints that cannot be resolved at bootstrap. It evaluates the decoded event identity, command name, user identity, and selected typed arguments. Suppression takes precedence over positive matches; an admitted event continues through the shared data path, whereas a rejected event reaches neither raw-event output nor detectors. In the absence of policies this boundary is permissive. Policies do not create one isolated detector engine per policy, and their descriptive mode and intent do not currently imply different execution pipelines. They instead compile into one effective collection plan, one selected detector set, and one event

admission boundary.

3.6.2 Point and Collective Detectors

The terms *point* and *collective* describe how much event evidence a detector requires. They do not denote learned anomaly models in this system. A point detector evaluates one observation, whereas a collective detector evaluates whether several observations satisfy an explicitly configured local sequence. Both forms are deterministic and rule based; neither learns a baseline or assigns a statistical anomaly score.

The selected detector definitions are compiled into a runtime registry. This registry owns a deterministic set of detector instances, while a separate dispatch index maps each event type to the detectors that consume it. The index is built once and keeps the hot path proportional to the interested detectors rather than to the complete catalogue. A failure produced by one detector is returned with that detector’s identity and does not prevent other candidates from processing the same event.

Every admitted event is passed to this layer in the same decoded representation used by policy matching; there is no detector-specific event conversion. A point detector applies deterministic predicates to one such event and emits a result immediately when they hold. A collective detector applies predicates to an ordered sequence and therefore owns bounded partial-match state. Both are instances of the same detector contract and share lifecycle, dispatch, error, metric, and alert handling. Their architectural difference is state ownership, not a separate processing pipeline.

Detector declarations describe consumption requirements rather than directly attaching probes. The policy and probe-planning stages must make the required observations available before the registry is built. Once monitoring begins, the detector engine does not modify the collection plan. This preserves a one-way dependency from collection planning to analysis and prevents a detector from changing kernel instrumentation while events are being processed.

3.6.3 Bounded Local Correlation

For a collective detector, the bootstrap phase compiles both its ordered steps and its correlation strategy. At runtime an admitted event is first

associated with one or more local correlation keys. It may advance the next expected step of an existing partial match; if it instead satisfies the first step, it starts or replaces the partial match for that key. Completion produces one alert from the accumulated evidence. Sequence order is therefore user-space arrival order: the engine neither searches historical output nor reorders events to infer a different causal sequence.

The correlation plan determines which observations may share state. Process correlation combines the host process identifier with process start time, so a reused numeric identifier does not join separate lifetimes. Process-tree correlation additionally permits an immediate parent-to-child transition, but does not build or traverse a persistent ancestry graph. Resource correlation uses the device and inode exposed by compatible file events, thereby identifying a local file object without treating a pathname as stable identity. Cgroup correlation uses the kernel cgroup identifier as a node-local execution-domain key and does not infer Kubernetes workload or cluster identity. When several dimensions are configured, their identities form a conjunctive composite key; every component must be available and agree.

State is deliberately detector-local and bounded. Each key identifies at most one incomplete sequence, each sequence has a short maximum lifetime, and each detector has a finite state capacity. Expiration is checked as relevant events are processed and when detector state is flushed; capacity pressure removes the oldest incomplete state. A requested correlation dimension is rejected during bootstrap if the event model cannot support it reliably. This is why user session correlation is not approximated from UID: doing so would silently join unrelated sessions. The design supports short point-to-point and collective relationships without becoming a persistent process graph or a general-purpose complex-event processing system [16, 1].

3.6.4 Alert Evidence and MITRE ATT&CK Metadata

An alert is the stable boundary between detection and the presentation path described in Section 3.5.3. It contains the detector identity and severity together with the evidence that completed the match: one event for a point detector or the ordered event set for a collective detector. Before forwarding the result, the engine may suppress an equivalent alert repeated within a short interval. This post-detection control does not alter detector state, event admission, or raw-event collection.

MITRE ATT&CK metadata belongs to the detector definition and crosses

two architectural boundaries. During bootstrap, policies may use tactics and techniques to select a relevant subset of the detector catalogue. After a match, the same metadata accompanies the alert and its evidence, providing a shared vocabulary for interpreting and organising the observed behaviour [28]. This makes the classification operational rather than purely documentary, while leaving the event predicates and correlation logic explicit in the detector itself.

The runtime validates the expected form of tactic and technique identifiers but does not depend on an online ATT&CK catalogue. Semantic review of mappings and evaluation of their representative coverage therefore remain part of the detector-authoring and experimental processes. Policy names are not currently attached to emitted alerts, because several policies are compiled into a shared execution plan rather than preserved as separate runtime branches.

3.7 Design Decisions and Scope Boundaries

The architecture resolves three principal trade-offs. The first is between observation breadth and target reliability. Selective loading, explicit probe dependencies, and validation against the deployed enterprise kernel are preferred to assuming that every registered event can be observed portably. The resulting design supports the stated Rocky Linux target without claiming universal compatibility.

The second trade-off is between early reduction and analytical flexibility. Small, verifier-compatible filters may prevent irrelevant records from crossing the kernel boundary, but policy interpretation and detector semantics remain in Go so that they can evolve independently from eBPF compilation. The explicit event contract makes this division possible, while the selected perf-event transport reflects the capabilities validated for the reference environment.

The third trade-off is between richer security interpretation and predictable runtime state. Deterministic point and bounded collective detectors provide more focused results than raw telemetry alone without introducing persistent graphs, statistical learning, or unbounded correlation. The prototype is therefore an observation and detection system rather than an enforcement or general-purpose event-processing engine. Its claims are limited to the event coverage, local relationships, ATT&CK mappings, compatibility, and

performance profiles evaluated in Chapter 6.

Container packaging supports later operational use but is a user-space and deployment concern described in Chapter 5. A wider platform may eventually aggregate alerts from several monitored hosts and add deployment context, but that external capability is neither implemented nor evaluated as part of this thesis. Within its stated boundary, the proposed system provides a coherent path from selected kernel observations to normalised telemetry and evidence-preserving local detections on the identified enterprise kernel.

Chapter 4

Kernel-Space Event Collection Implementation

This chapter describes how the kernel-space collection design developed in Chapter 3 is realised in the prototype. It covers the build artefacts, shared event-construction mechanisms, implemented hook families, binary record contract, perf-buffer transport, early UID admission and compatibility work required by the target Rocky Linux kernel. Chapter 5 follows the record into the Go runtime and describes loading, decoding, policy evaluation, detection and presentation. Experimental behaviour and performance measurements are reserved for Chapter 6.

4.1 Build Toolchain and Project Organisation

The build process consists of three ordered stages. First, the libbpf source in `3rdparty/libbpfgo/libbpf/src` is compiled locally into the static library `dist/libbpf/obj/libbpf.a`, together with the required headers and package metadata. Second, Clang compiles the eBPF source into `dist/project.bpf.o` for the x86-64 target. Finally, the Go application is built through CGO using the local libbpf library and libbpfgo bindings, producing the executable `dist/project`. The Makefile preserves this order and ensures that all stages use the dependencies included in the repository rather than system-installed versions.

The eBPF object is also embedded in the Go executable. For this rea-

son, `dist/project.bpf.o` must be generated before the Go build starts. At runtime, the application can load the embedded object without requiring a separate copy beside the executable. Developers may still provide an external object through the command-line configuration for testing or compatibility analysis. The executable nevertheless remains dependent on the host kernel, its BTF data, the required privileges and the system libraries used by CGO.

4.2 eBPF Program Implementation

The eBPF object contains programs attached to different kernel hook types. Most of these programs produce records in one of two ways. A direct producer creates and submits an event during a single hook invocation. A paired producer saves the request arguments when an operation begins and completes the event when a return hook provides its result.

Physical hooks and public events do not have a one-to-one relationship. Several eBPF programs may contribute to the same logical event and use the same event schema. Conversely, some attached programs only provide internal support and do not generate a public event. Section 5.1 explains how the loader translates these relationships into an attachment plan.

4.2.1 Shared Event-Construction Framework

Both construction forms use a per-CPU workspace to build events. Each CPU has its own reusable record, avoiding the need to place the complete event structure on the limited eBPF stack. Additional per-CPU maps provide temporary storage for path reconstruction and intermediate kernel objects, while task and process maps retain context that can be reused across hook invocations.

The common initialisation helper clears the argument buffer and records the basic identity and timing information. Each hook then appends its event-specific fields. Immediately before submission, the common submission helper updates the complete task context and sends the record through the kernel-to-user-space interface described in Section 4.3.

Listing 4.1 shows the main stages shared by these helpers. Details related to process-map initialisation and optional stack capture are omitted because they do not change the event-construction process.

```
1  if (!p->event)
2      p->event = bpf_map_lookup_elem(&event_data_map, &zero);
3  if (!p->event)
4      return 0;
5
6  p->config = bpf_map_lookup_elem(&config_map, &zero);
7  if (!p->config)
8      return 0;
9
10 reset_event_args_buf(p->event);
11 p->event->task = (struct task_struct *) bpf_get_current_task();
12
13 u64 id = bpf_get_current_pid_tgid();
14 p->event->context.task.host_tid = id;
15 p->event->context.task.host_pid = id >> 32;
16 p->event->context.eventid = event_id;
17
18 init_task_context(&p->event->context.task,
19                  p->event->task, p->config->options);
20
21 u32 size = sizeof(event_context_t) + sizeof(u8) +
22           p->event->args_buf.offset;
23 long ret = bpf_perf_event_output(
24     p->ctx, &events, BPF_F_CURRENT_CPU, p->event, size);
25
26 update_event_stats(p->event->context.eventid, ret);
27 return ret;
28
```

Listing 4.1: Shared event construction and submission path.

The per-CPU workspace is reused after each event submission. When the submission helper is called, the completed portion of the record is copied to the perf-event channel, so later modifications to the workspace do not affect the submitted event. If the program cannot obtain the required workspace, configuration or task context, it stops processing that invocation without producing a record.

The complete implementation also verifies the record size before submission and optionally updates delivery statistics. The listing therefore focuses on the point at which the event leaves kernel space. The precise size checks and error-handling rules are described in Section 4.3.

Paired producers build one event across two hook invocations. The entry program stores the request arguments in a shared map, while the return program retrieves them and adds the operation result. Each map entry is

identified by the logical event and the host thread that issued the request. This allows the return hook to recover the correct arguments without mixing different operations performed by the same thread.

```
1  statfunc u64 get_args_id(u32 event_id)
2  {
3      return ((u64) event_id << 32) |
4             (u32) bpf_get_current_pid_tgid();
5  }
6
7  statfunc int save_args(args_t *args, u32 event_id)
8  {
9      u64 key = get_args_id(event_id);
10     bpf_map_update_elem(&args_map, &key, args, BPF_ANY);
11     return 0;
12 }
13
14 statfunc int load_args(args_t *args, u32 event_id)
15 {
16     u64 key = get_args_id(event_id);
17     args_t *saved = bpf_map_lookup_elem(&args_map, &key);
18     if (!saved)
19         return -1;
20
21     /* Fixed argument-slot copies are omitted. */
22     return 0;
23 }
```

Listing 4.2: Saving and recovering arguments for paired producers.

Including the event identifier in the key prevents different paired operations from using the same map entry. The implementation copies the seven argument slots explicitly, keeping every memory access statically bounded for the eBPF verifier. If no entry is found, the return program cannot reconstruct the event and therefore produces no record.

After the saved arguments have been copied, the return program removes the map entry and appends the operation result. New entry data is stored with `BPF_ANY`, allowing it to replace stale state left by a missing return hook. The map is therefore temporary correlation storage, not a history of process activity.

The following subsections apply these two construction patterns to different hook families. The interpretation of each event depends on where the hook is placed within the kernel operation. A syscall-entry hook records the operation requested by a process, but not its outcome. A security hook observes

the operation while it is being checked by the kernel, without necessarily revealing the final decision. A committed-state hook runs after a new state has been installed, while a lifecycle tracepoint reports that a process transition has occurred. An exit or return hook completes the observation by recording the result available at that boundary, such as a returned value or error code.

4.2.2 Process Lifecycle and Executable Transitions

Process monitoring must distinguish between a creation request and the lifecycle transition produced by the kernel. Syscall events expose the operation selected by the process and the result returned at the syscall boundary. Scheduler tracepoints instead observe tasks that have already been created, renamed or terminated. The implementation combines these sources because neither view is sufficient on its own.

4.2.2.1 Process Creation and Lifecycle

The `fork` and `vfork` events provide the simplest syscall-level view of process creation. Their schemas contain only the return value, so their entry probes do not retain any arguments. A positive result identifies the child returned to the caller, while a negative result records the reported error.

The `clone` event captures a more configurable creation request. Its entry probe retains the clone flags, stack address, parent and child thread-ID pointers, and thread-local-storage value. These fields describe which resources the new task should share with its parent and where its execution context should be initialised. The exit probe combines them with the returned child identifier or error. The resulting event therefore preserves both the requested creation model and the syscall outcome.

Syscall results alone do not provide the complete parent-child state observed by the scheduler. The `sched_process_fork` tracepoint runs after the child task has been created and receives both task structures. It records the host PID and TID of the parent and child, together with the child's start time. Listing 4.3 shows the extraction of this effect-level evidence.

```
1 struct task_struct *parent =  
2     (struct task_struct *) ctx->args[0];  
3 struct task_struct *child =  
4     (struct task_struct *) ctx->args[1];
```

```
5
6 u32 parent_pid = get_task_host_tgid(parent);
7 u32 parent_tid = get_task_host_pid(parent);
8 u32 child_pid = get_task_host_tgid(child);
9 u32 child_tid = get_task_host_pid(child);
10 u64 child_start = get_task_start_time(child);
11
12 save_to_submit_buf(&p.event->args_buf,
13                  &parent_pid, sizeof(u32), 0);
14 save_to_submit_buf(&p.event->args_buf,
15                  &parent_tid, sizeof(u32), 1);
16 save_to_submit_buf(&p.event->args_buf,
17                  &child_pid, sizeof(u32), 2);
18 save_to_submit_buf(&p.event->args_buf,
19                  &child_tid, sizeof(u32), 3);
20 save_to_submit_buf(&p.event->args_buf,
21                  &child_start, sizeof(u64), 4);
```

Listing 4.3: Parent-child evidence collected by `sched_process_fork` (abridged).

The start time complements the numeric child identifiers. PIDs and TIDs may be reused after a task terminates, while the start time allows user space to distinguish separate task lifetimes associated with the same identifier. This combination also provides a stronger process identity for local detector correlation.

Two direct tracepoints cover later lifecycle changes. `task_rename` records the previous and new command names when the task name changes. This operation is kept separate from executable replacement because a command-name change does not imply that a new program image was installed. The `sched_process_exit` event records the task's exit code and whether the complete thread group is terminating, distinguishing a single thread exit from the end of the process.

The implementation emits these lifecycle relations as bounded events rather than maintaining a persistent process graph in kernel space. Syscall events describe requests and returned outcomes, whereas scheduler tracepoints provide evidence of task states visible after the corresponding kernel transition. Longer-lived reconstruction remains a user-space responsibility.

4.2.2.2 Executable Transitions

Executable-transition monitoring must preserve both the program requested by a process and the outcome of the replacement operation. The pathname and arguments available at syscall entry describe the process's intention, but they cannot establish whether the kernel accepted the executable or installed the new image. The implementation therefore observes the request, binary security processing and final outcome at separate kernel boundaries.

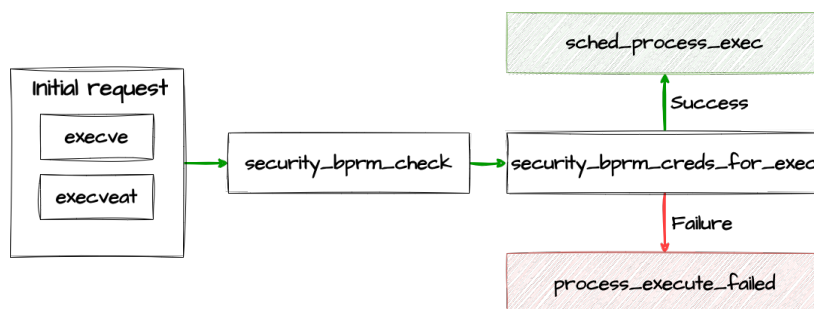


Figure 4.1: Events used to observe execution requests, kernel security processing and executable-transition outcomes.

Figure 4.1 represents monitoring coverage rather than a guaranteed one-to-one event sequence. Binary-handler and interpreter processing may cause security-stage observations to occur more than once. Furthermore, the security hooks identify intermediate kernel processing but do not expose the final security decision.

The initial request is captured through `execve` and `execveat`. Both events preserve the requested pathname and up to 38 argument strings while the original user-space memory is still available. The `execveat` event additionally records the directory file descriptor and execution flags, which are required to interpret relative paths and descriptor-based execution. These events are emitted at syscall entry and therefore do not contain an execution result.

The `security_bprm_check` hook observes the executable after the kernel has constructed its `linux_binprm` representation. At this stage, the implementation can record the resolved pathname, device and inode, original filename, argument count and environment count. These fields associate the request with a resolved kernel object rather than relying only on the

caller-supplied path.

The `security_bprm_creds_for_exec` hook provides a related view of credential preparation. It captures the same binary identity fields and adds the file change time. This observation does not prove that the prepared credentials became active; committed credential transitions are monitored separately by the mechanism described in Section 4.2.3.1.

The final outcome is represented by two mutually exclusive logical events. A `sched_process_exec` event confirms that the process installed a new executable image and records its resolved filename. An unsuccessful `execve` or `execveat` instead produces `process_execute_failed`, which retains the operation variant, directory file descriptor, requested pathname, flags and negative return value.

The failure event uses paired entry and exit programs because the pathname and execution options are available before the syscall runs, while the error is available only when it returns. Listing 4.4 shows the return-side selection that suppresses successful executions.

```
1  args_t args = {};  
2  if (load_args(&args, PROCESS_EXECUTE_FAILED) != 0)  
3      return 0;  
4  
5  del_args(PROCESS_EXECUTE_FAILED);  
6  
7  if (ret >= 0)  
8      return 0;  
9  
10 program_data_t p = {};  
11 if (!init_program_data(&p, ctx, PROCESS_EXECUTE_FAILED))  
12     return 0;  
13  
14 int operation = (int) args.args[0];  
15 int dirfd = (int) args.args[1];  
16 const char *pathname = (const char *) args.args[2];  
17 int flags = (int) args.args[3];  
18  
19 save_to_submit_buf(&p.event->args_buf,  
20                  &operation, sizeof(operation), 0);  
21 save_to_submit_buf(&p.event->args_buf,  
22                  &dirfd, sizeof(dirfd), 1);  
23 if (pathname)  
24     save_str_to_buf(&p.event->args_buf, pathname, 2);  
25 save_to_submit_buf(&p.event->args_buf,  
26                  &flags, sizeof(flags), 3);  
27 save_to_submit_buf(&p.event->args_buf,
```

```
28         &ret, sizeof(ret), 4);  
29  
30 return events_perf_submit(&p);
```

Listing 4.4: Return-side emission of failed executable transitions (abridged).

The saved entry state is removed before the return value is evaluated. A successful result therefore clears the temporary map entry without generating a failure record, while a negative result is serialised with the original request. Separate entry and exit programs handle `execve` and `execveat`, but all four physical programs converge on the same public event schema.

This design keeps caller intent, resolved binary identity and observed outcome as distinct forms of evidence. It avoids interpreting an execution attempt as a successful transition and allows user space to present negative return values as readable execution errors.

4.2.3 Identity, Credentials, Process Control and Limits

4.2.3.1 Identity and Credentials

A privilege transition cannot be reconstructed reliably from a set-ID syscall alone. The syscall identifies the change requested by the process and reports its result, but credentials may also change during executable loading or through internal kernel paths. Conversely, a security hook exposes a proposed transition without proving that the new state became active. The implementation therefore separates requested identity changes, security mediation, committed credentials and subsequent capability checks.

Linux associates each process with several user and group identifiers. The real ID represents the identity under which the process was started, while the effective ID is used for most access-control decisions. The saved ID allows a process to retain an effective identity that may later be restored, supporting temporary privilege reduction and recovery. Linux also maintains filesystem IDs, which are used specifically during filesystem permission checks. The same distinction applies to user and group credentials [17].

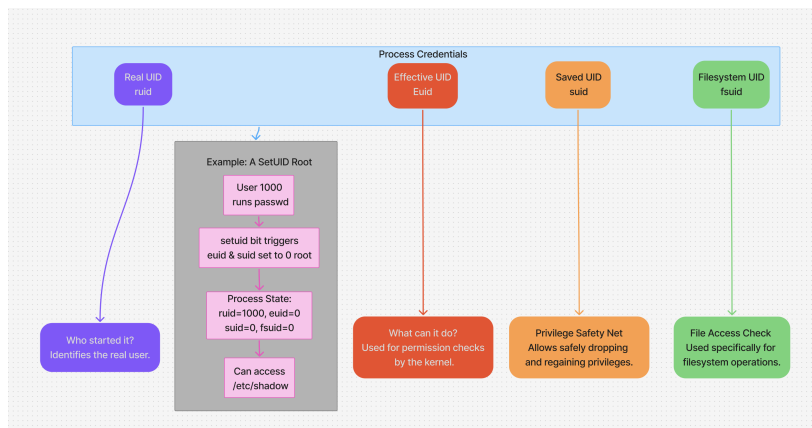


Figure 4.2: Real, effective, saved and filesystem UIDs for a process [17].

Monitoring all these values is security-relevant because a process may change its effective or filesystem privileges without changing its real identity. Saved identities may also allow privileges to be regained after an apparent drop. Observing only the real ID would therefore hide important transitions in the process's effective authority [17].

Explicit identity requests are captured through the `setuid`, `setgid`, `setreuid`, `setregid`, `setresuid`, `setresgid`, `setfsuid` and `setfsgid` events. Their entry programs retain one, two or three identifiers according to the syscall interface. This preserves the distinction between single-ID changes, separate real and effective identities, complete real/effective/saved transitions and filesystem-specific identities. The exit programs append the value returned to the caller.

The return value cannot be interpreted uniformly across the family. Most set-ID operations use a conventional status value, while `setfsuid` and `setfsgid` return the previous filesystem identity. Preserving the originating logical event is therefore necessary for user space to interpret the result correctly.

The `security_task_fix_setuid` hook adds the mediation-stage view. It records the old and proposed real, effective, saved and filesystem UIDs, together with a flag that distinguishes the set-ID operation category. The eBPF program compares the two credential proposals and suppresses the event when none of the monitored UID fields changes. Since the hook is observed before installation and its result is not captured, the event describes

the transition examined by the security subsystem rather than a confirmed change.

No public `security_task_fix_setgid` event is currently exposed. Group-ID requests remain visible through the set-group syscall events, while changes that become active are included in the broader `commit_creds` event.

The `commit_creds` hook observes the point at which replacement credentials become active for the current task. It reads both the currently installed credentials and the replacement supplied by the kernel, converting them into fixed-width snapshots. Each snapshot contains real, effective, saved and filesystem user and group identifiers, the user-namespace identifier, secure bits, and the inheritable, permitted, effective, bounding and ambient capability sets.

Credential installation is a frequent kernel operation, and not every invocation changes a field relevant to the monitor. The implementation therefore compares the two snapshots before submission. Listing 4.5 shows this change-only emission strategy.

```
1 slim_cred_t old_slim = {};
2 slim_cred_t new_slim = {};
3
4 /* Identifiers, namespace, secure bits and capability
5  * sets are copied into the two snapshots. */
6
7 if (old_slim.uid == new_slim.uid &&
8     old_slim.gid == new_slim.gid &&
9     old_slim.euid == new_slim.euid &&
10    old_slim.egid == new_slim.egid &&
11    /* Remaining monitored fields are compared here. */
12    old_slim.cap_ambient == new_slim.cap_ambient)
13     return 0;
14
15 save_to_submit_buf(&p.event->args_buf,
16                  &old_slim, sizeof(slim_cred_t), 0);
17 save_to_submit_buf(&p.event->args_buf,
18                  &new_slim, sizeof(slim_cred_t), 1);
19
20 return events_perf_submit(&p);
```

Listing 4.5: Kernel-side suppression of unchanged credential snapshots (abridged).

The comparison occurs before the event enters the perf-event channel. Unchanged credential installations therefore consume neither transport band-

width nor user-space decoding work. When a relevant difference is found, both snapshots are preserved, allowing the transition to be analysed without reconstructing the previous state from an earlier event.

The final observation point is `cap_capable`. This event records the capability number presented during a kernel capability check, exposing the use of privilege-sensitive operations even when no credential replacement occurs. A capability check is not equivalent to a privilege change or a successful operation: the current schema does not retain the checked credentials, user namespace, option flags or final decision.

Together, these events distinguish four forms of evidence: an explicit identity request, a proposed transition under security mediation, the credential state installed by the kernel and a later capability check. This prevents the monitoring pipeline from treating all privilege-related activity as the same kind of event.

4.2.3.2 Process Control and Limits

Process-control operations can alter another task, modify the behaviour of the current process or change the limits within which it executes. These operations are security-relevant because they may support inspection, interference, persistence or resource manipulation. The implementation observes them at different boundaries according to where their most useful evidence becomes available.

Cross-process control is represented by `ptrace` and `security_task_kill`. The `ptrace` entry program retains the request code, target PID and the address and data arguments supplied by the caller. These pointer values are deliberately treated as opaque: the eBPF program does not copy the memory to which they refer. At syscall exit, the stored request is combined with the return value, allowing user space to distinguish an accepted request from an error.

Signal delivery is observed through `security_task_kill`. The common event context already identifies the source task, while the event-specific payload describes the destination through its host PID and TID, UID and command name. The signal number completes the source-operation-target relation. Signal zero is retained because it can be used to test whether a target exists and whether the caller has permission to signal it, even though no signal is actually delivered.

Listing 4.6 shows how the destination identity is extracted from the target task rather than inferred from a caller-supplied numeric PID.

```
1  u32 target_pid = get_task_host_tgid(target);
2  u32 target_tid = get_task_host_pid(target);
3  u32 target_uid = 0;
4  char target_comm[TASK_COMM_LEN] = {};
5
6  const struct cred *target_cred =
7      get_task_real_cred(target);
8  if (target_cred)
9      target_uid = BPF_CORE_READ(target_cred, uid.val);
10
11  if (bpf_probe_read_kernel_str(
12      target_comm, sizeof(target_comm),
13      target->comm) <= 0)
14      return 0;
15
16  save_to_submit_buf(&p.event->args_buf,
17                    &target_pid, sizeof(u32), 0);
18  save_to_submit_buf(&p.event->args_buf,
19                    &target_tid, sizeof(u32), 1);
20  save_to_submit_buf(&p.event->args_buf,
21                    &target_uid, sizeof(u32), 2);
22  save_str_to_buf(&p.event->args_buf, target_comm, 3);
23  save_to_submit_buf(&p.event->args_buf,
24                    &sig, sizeof(int), 4);
```

Listing 4.6: Target evidence collected by `security_task_kill` (abridged).

Reading the destination directly from its kernel task structure avoids relying only on an identifier supplied by user space. However, this hook runs during security mediation and does not expose the final decision or confirm that the signal was delivered. It records an attempted relation rather than a completed state change.

Changes to the current task’s behaviour are monitored through `security_task_prctl` and `do_sigaction`. The former records the requested `prctl` option and its four machine-word arguments. The eBPF program intentionally leaves option-specific interpretation to user space, avoiding a large command-dependent parser in kernel space.

The `do_sigaction` event focuses specifically on signal-handler state. It records the signal number, whether a new action was supplied and whether the caller requested the previous action. For both the proposed and currently installed actions, it captures the flags, first signal-mask word and handler

category or address. This makes changes such as installing, replacing or ignoring a handler observable. Since the hook is attached at function entry, the event does not confirm that the proposed action was installed.

Resource-limit monitoring combines a mediation view with syscall-level outcomes. The `security_task_setrlimit` hook receives the resolved target task and proposed kernel `rlimit` structure. It records the target host PID, resource identifier, soft limit and hard limit while the operation is being checked. It does not include the final decision.

The paired `prlimit64` event instead preserves the target PID, resource identifier, addresses of the new and old user-space limit structures, and the syscall return value. The pointed-to structures are not dereferenced by this event. These two observations are therefore complementary: the security hook exposes the proposed kernel values, while the syscall event preserves the request interface and its reported outcome.

Finally, `security_settime64` observes attempts to modify the system clock. It records the proposed seconds and nanoseconds together with the time-zone offset and daylight-saving field. Changing wall-clock time is privilege-sensitive and can affect timestamp-based analysis, for example an attacker can leverage the time modification to obscure their activities or perform some time based TOTP authentication, but this mediation-stage event does not by itself establish that the new time was installed.

This design keeps cross-process relations, task configuration and resource changes as separate forms of evidence. Syscall return events provide explicit outcomes where available, while security hooks expose resolved kernel objects and proposed values without being treated as confirmation of success.

4.2.4 File and Filesystem Security

Filesystem monitoring must distinguish the name supplied by a process from the object eventually reached by the kernel. A pathname may be relative, interpreted from a directory file descriptor or contain symbolic links. The Virtual File System resolves these elements before operating on the corresponding filesystem object [47].

The implementation therefore combines two complementary perspectives. Syscall probes preserve the original request and its returned result, while security hooks observe objects that have already undergone kernel pathname resolution. Where available, the resolved device and inode numbers provide

a local object identity, since an inode number is unique only within its filesystem [26].

The **open** event provides the syscall-level perspective. Entry probes for **open** and **openat** retain the operation variant, directory file descriptor, requested pathname, flags and creation mode. Exit probes add either the returned file descriptor or a negative error. The two syscall variants therefore share one logical schema without losing the differences between their calling conventions.

The same design is used for filesystem metadata changes. The **chmod** event monitors requests to modify permission bits, which determine whether an object may be read, written or executed. It combines **chmod**, **fchmod** and **fchmodat**, recording the relevant pathname, descriptor or directory-relative context, the requested mode and the syscall result.

The **chown** event monitors requests to replace the user or group ownership of an object. It combines **chown**, **fchown**, **fchownat** and **lchown**, retaining the requested owner and group identifiers, pathname or descriptor context, flags and returned result. The operation identifier remains important because **lchown** changes the ownership of a symbolic link itself rather than following the link to its target.

This consolidation separates the public event model from physical attachment. Six entry and exit programs produce the single **chmod** event, while eight produce **chown**. A compact operation field identifies the originating syscall and avoids duplicating otherwise similar event schemas.

The syscall view is complemented by **security_file_open**. This hook is invoked while the kernel is opening a file, after the Virtual File System has resolved the pathname and constructed a **struct file**. Linux Security Modules may use this stage to preserve open-time security state and re-evaluate conditions previously checked through **inode_permission** [50]. The hook therefore exposes the resolved object under security inspection, but does not report the final LSM decision or syscall result.

From the supplied **struct file**, the program obtains the resolved pathname, file flags, device number, inode number and change time. When the originating syscall is recognised, it also attempts to recover the pathname argument from the current task register state. Listing 4.7 shows the essential separation between these two forms of evidence.

```
1 void *resolved = get_path_str(&file->f_path);
2 dev_t device = get_dev_from_file(file);
3 unsigned long inode = get_inode_nr_from_file(file);
4
5 void *requested = "";
6 struct pt_regs *regs = get_current_task_pt_regs();
7
8 if (p.event->context.syscall == SYSCALL_OPEN)
9     requested = get_syscall_arg1(p.event->task, regs, false);
10 else if (p.event->context.syscall == SYSCALL_OPENAT)
11     requested = get_syscall_arg2(p.event->task, regs, false);
12
13 save_str_to_buf(&p.event->args_buf, resolved, 0);
14 save_to_submit_buf(&p.event->args_buf, &device, sizeof(device), 2);
15 save_to_submit_buf(&p.event->args_buf, &inode, sizeof(inode), 3);
16 save_str_to_buf(&p.event->args_buf, requested, 5);
```

Listing 4.7: Requested and resolved evidence collected by `security_file_open`.

The requested and resolved paths answer different questions. The first describes how the process referred to the file; the second describes the path reconstructed from the kernel object. They may differ because of symbolic links, relative traversal or mount topology. Requested-path recovery is best-effort and may produce an empty value when the syscall or register state cannot be identified. The resolved object remains available directly through the `struct file`.

Path reconstruction is itself constrained by the eBPF execution model. The implementation walks kernel dentries backwards and assembles the path in per-CPU scratch storage rather than on the restricted eBPF stack. The traversal is bounded to 20 components and a 4096-byte working region, allowing the verifier to establish finite execution and memory bounds. Deep or disconnected paths may consequently produce a partial or explicitly disconnected representation. A resolved pathname must therefore be understood as bounded kernel evidence rather than an unconditional canonical path.

Device and inode values provide a more stable local correlation key than a pathname alone. Detectors can associate observations generated by different processes when they refer to the same `device:inode` pair, even if the path strings differ. The pathname remains useful as a detector condition, but does not define resource identity. This correlation is deliberately local and time-bounded because inode values can eventually be reused and layered filesystems may expose different identities for apparently equivalent paths.

Further file hooks observe access checks and control requests on resolved objects. `security_file_permission` records the pathname, device-inode identity, change time and permission mask presented during a read, write or execution check. It shows that the check occurred, but does not establish whether access was ultimately granted. This is also a high-volume observation point, making selective event activation particularly important.

The `security_file_ioctl` event monitors file and device-specific control requests. It records the resolved target identity, ioctl command and argument value. The argument is intentionally retained as an opaque value: its interpretation depends on both the command and the target device, and decoding every possible structure inside eBPF would increase complexity and verifier cost.

Directory-entry hooks preserve objects at the point where their namespace representation may change. `security_inode_unlink` records the path, device, inode and change time of an entry proposed for removal. Capturing this identity before the unlink operation is valuable because the pathname may no longer resolve afterwards.

The `security_inode_rename` event records the old and new paths, rename flags and the identity of the source object. Its implementation also reveals a consequence of bounded per-CPU storage. Both paths are reconstructed through the same scratch buffer, so the source path must be serialised into the event before reconstruction of the destination begins. Otherwise, the second traversal would overwrite the first result.

Creation-related hooks retain evidence specific to the new directory entry. `security_inode_symlink` records the pathname of the symbolic link and the caller-supplied target string, which may be relative or may not yet identify an existing object. `security_inode_mknod` records the pathname, requested mode and device number of a new special node. The mode identifies the object type and permission bits, while the device number is meaningful for character and block devices. Both events observe security mediation and do not contain a final syscall result.

Mount hooks extend monitoring from individual objects to filesystem topology. `security_sb_mount` records the source name, resolved target path, filesystem type and mount flags. The filesystem-specific data argument is not copied because its format varies between filesystems and may contain sensitive configuration. `security_sb_umount` records the resolved mount-point, filesystem type and unmount flags. As entry observations of security

hooks, these events show that the requested topology change reached mediation, not that it completed successfully.

The resulting implementation preserves three distinct forms of filesystem evidence: the operation requested by a process, the kernel object involved in security mediation and, for paired syscall probes, the result returned to the caller. Keeping these stages separate prevents a pathname request or an LSM observation from being presented as proof that the corresponding filesystem operation succeeded.

4.2.5 Memory, Namespace and Cgroup Transitions

Memory, namespaces and cgroups control different aspects of process execution. Memory mappings determine which regions a process may access and how they may be used. Namespaces determine which kernel resources a process can observe, while cgroups define its position within a resource-control and accounting hierarchy. Monitoring these mechanisms is security-relevant because they are involved in code loading, process-memory access and changes to isolation boundaries.

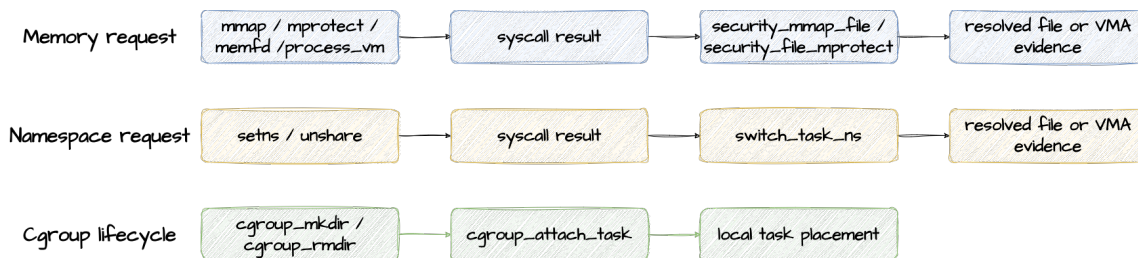


Figure 4.3: Observation points used for memory operations, namespace transitions and local cgroup changes.

Figure 4.3 represents complementary observation paths rather than a mandatory sequence. A memory syscall may not reach the corresponding security hook, and a successful namespace-related syscall does not necessarily modify every namespace represented by the committed-state event.

The `memfd_create` event monitors the creation of an anonymous, RAM-backed file. Such an object initially has no conventional filesystem pathname, but it may later be mapped into memory or used as an executable

image. The event records the requested name and creation flags, followed by the returned file descriptor or a negative error. This makes successful anonymous-file creation distinguishable from a failed attempt without requiring the monitor to inspect its future contents.

The `mmap` event records requests to create or modify mappings in a process address space. Its entry probe retains the requested address, length, protection mask, mapping flags, file descriptor and offset. The exit probe adds either the address selected by the kernel or a negative error. These fields distinguish anonymous mappings from file-backed mappings and identify requests that combine writable and executable permissions.

The `mprotect` event monitors requests to change the permissions of an existing memory range. It records the starting address, length, requested protection mask and returned status. The related `pkey_mprotect` event additionally records the protection key assigned to the range. Protection-key support is dependent on the target architecture and kernel configuration, so the returned value remains necessary for distinguishing an applied request from an unsupported or rejected operation.

The syscall events describe the process request, but they do not identify the resolved kernel objects examined during security mediation.

The `security_mmap_file` hook complements `mmap` for file-backed mappings. It records the resolved pathname, file flags, device, inode, change time, requested protections and mapping flags. Anonymous mappings are omitted because no backing `struct file` is available; they remain visible through the syscall event.

The `security_file_mprotect` hook observes a protection change through the virtual memory area supplied by the kernel. It preserves three distinct protection values: the permissions requested by user space, the permissions adjusted by the kernel for security evaluation, and the access bits currently installed on the region. It also records the region start and length. When the mapping is file-backed, the event adds its resolved pathname and device-inode identity; anonymous regions retain the protection and address evidence without file metadata.

This distinction makes the event more informative than the syscall request alone. It can show, for example, that a previously writable region is being evaluated for executable access and whether that region is associated with a resolved file. Since the probe observes entry to the security hook, however, it does not contain the hook decision or the final syscall result.

The implementation reads the supplied virtual memory area directly. It does not perform an independent search through the process memory-management tree. This keeps the collection path bounded and avoids duplicating work already performed by the kernel before invoking the security hook.

Cross-process memory access is monitored through `process_vm_readv` and `process_vm_writev`. These events record the target PID, addresses and element counts of the local and remote I/O-vector arrays, operation flags and the number of bytes transferred or the returned error. The vector elements and transferred memory contents are not copied. This retains evidence that one process accessed another process's address space while limiting overhead and avoiding collection of potentially sensitive payload data.

Namespace events monitor changes to the resource views associated with a process. The `setns` syscall requests that the caller join an existing namespace referenced by a file descriptor. Its event records the descriptor, the requested namespace type and the returned result. A type value of zero allows the kernel to infer the namespace type from the descriptor.

The `unshare` syscall instead requests that selected execution-context resources cease to be shared with other processes. Its event records the complete `CLONE_*` flag mask and the syscall result. Together, `setns` and `unshare` distinguish entry into an existing namespace from the creation or separation of an execution context.

A successful syscall result confirms that the request was accepted, but does not directly expose the identifiers installed in the task. The `switch_task_ns` event supplies this state-level perspective by observing `switch_task_namespaces`, where the kernel replaces the task's namespace proxy.

The hook compares the old and proposed namespace sets and emits an event only when at least one monitored identifier changes. The record contains the task PID and only the new identifiers for the changed mount, PID-for-children, UTS, IPC, network or cgroup namespaces. Unchanged fields are omitted from the indexed argument stream, keeping the record sparse and suppressing no-op transitions. User namespaces are not part of this namespace-proxy comparison; credential-related user-namespace changes are observed separately through the credential hooks.

Cgroup monitoring addresses task placement rather than namespace replacement. A cgroup namespace changes how a hierarchy is presented to a process, whereas cgroup attachment changes the hierarchy node to which the task belongs. The implementation preserves this distinction.

The `cgroup_attach_task` raw tracepoint reports movement of a task to a destination cgroup. It records the destination cgroup and hierarchy identifiers, the supplied cgroup path, target command name, host PID and TID, and whether the operation concerns the complete thread group. This allows the monitor to observe containment changes even when the task identity itself remains unchanged.

The `cgroup_mkdir` and `cgroup_rmdir` events describe the lifecycle of the local cgroup hierarchy. Both retain the cgroup identifier, bounded path and hierarchy identifier supplied at creation or removal. They do not reconstruct a persistent hierarchy in user space; each event represents a local lifecycle observation.

Cgroup identifier extraction also contains a compatibility decision specific to the supported kernel range. The layout of the underlying `kernfs_node` identifier differs between older kernels, newer kernels and the hidden KABI representation used by the target RHEL-derived kernel. CO-RE field-existence checks select the available layout at load time, allowing the same helper to obtain the identifier without hard-coding a single structure version.

Namespace inode numbers and cgroup identifiers remain local kernel identities. They are useful for comparing events and for detector correlation on one node, but they are not interpreted as pod, container or cluster-wide workload identifiers. The implementation therefore provides evidence of local memory and isolation transitions without claiming to reconstruct a persistent namespace graph, cgroup hierarchy or Kubernetes execution model.

4.2.6 Kernel Integrity and Kernel-Facing Activity

Kernel modules, eBPF objects and dynamic instrumentation can modify or inspect behaviour inside the kernel. These mechanisms are legitimate parts of Linux, but they also expose high-impact interfaces whose use is relevant to runtime security monitoring. The implemented hooks preserve the operation, object identity and result available at each observation point. They provide evidence for subsequent detectors and do not classify an operation as malicious by themselves.

Kernel-module monitoring separates lifecycle notifications from module initialisation results. The `module_load` raw tracepoint records a module as it enters the kernel module lifecycle, while `module_free` reports its removal.

Both events preserve the module name and, when available, its version and source-version strings. They do not copy module code, sections or other internal contents.

These lifecycle events do not indicate whether the module's initialisation function completed successfully. The `do_init_module` event therefore uses a kprobe and kretprobe pair. At function entry, the module object is retained in per-thread temporary state. At return, the implementation retrieves that object, extracts the module identity and appends the function result. A zero value identifies successful initialisation, while a negative value preserves the reported kernel error.

This distinction prevents a module observed during loading from being automatically treated as operational. The lifecycle tracepoints identify additions and removals, whereas `do_init_module` provides the outcome of the initialisation stage. These events can later be correlated without assigning a security meaning to ordinary module-management activity.

The eBPF hooks observe a different kernel-facing interface. They are kprobes attached to Linux Security Module functions invoked while eBPF commands and objects undergo security checks. They are not themselves BPF LSM enforcement programs and do not modify the kernel's decision.

The `security_bpf` hook monitors operations entering the security path of the `bpf` syscall. Its purpose is to expose requests such as map creation, program loading and program attachment. The event records the BPF command and attribute size, but does not copy `union bpf_attr`, whose active fields depend on the command.

The `security_bpf_map` hook observes security checks involving an existing eBPF map. Since maps retain state and exchange data between kernel programs and user space, identifying access to them provides useful object-level evidence. The event records the map identifier and bounded name, without collecting its contents.

The `security_bpf_prog` hook observes security operations involving an existing eBPF program. It records the program identifier, name and type, allowing programs intended for kprobes, tracepoints, cgroups and other execution contexts to be distinguished.

Together, these hooks identify the requested BPF operation and the map or program involved in security mediation.

Dynamic instrumentation and kernel-facing interfaces expose activity that

is not always visible through ordinary process syscalls. The following events monitor selected interfaces used for tracing, symbol resolution, procfs creation, kernel-initiated execution and kernel file consumption.

The `register_kprobe` event monitors attempts to install dynamic kprobes. It records the target symbol, pre- and post-handler addresses, and registration result. This distinguishes successful installations from rejected requests, although the symbol may be absent when registration uses a kernel address directly.

The `kallsyms_lookup_name` event monitors kernel symbol resolution. It combines the requested symbol name with the returned address, where zero indicates that no address was found. It covers this specific lookup interface rather than every possible source of kernel addresses.

The `proc_create` event observes requests to create procfs interfaces. It records the entry name and proc-operations address, making kernel-created user-space control or information endpoints visible. It does not include the parent directory, complete path or creation result.

The `call_usermodehelper` event monitors requests by the kernel to start a user-space program. It records the executable path, bounded argument and environment arrays, and wait mode. The arrays contain at most 38 elements and remain subject to the event-size limit. The event represents the request, not successful execution or completion of the helper.

The `security_kernel_read_file` event observes the security stage before the kernel consumes a file. It records the resolved pathname, device, inode, change time and read purpose, distinguishing uses such as modules, firmware, policy data and kexec images.

The later `security_kernel_post_read_file` event records the same file identity and purpose together with the supplied byte size. This size is not a return value. The two stages are emitted independently: the implementation does not correlate them automatically, copy file contents or compute a digest.

Together, these hooks expose selected instrumentation, procfs, kernel-initiated execution and kernel-file activity. They do not inspect handler code, reconstruct every kernel relationship, prove integrity or enforce a security decision. Chapter 5 explains how the corresponding logical events activate their physical producers and support dependencies.

4.3 Binary Event Contract and Kernel Transport

Every event crosses the kernel boundary in the same binary format. The record begins with a 136-byte context containing common process, host and event metadata. It is followed by a one-byte argument count and by the arguments captured for that specific event. The Go decoder mirrors the order and width of the C context fields. Only the populated part of the argument buffer is submitted, so an event does not carry the complete 32,000-byte workspace used while it is being constructed.

Arguments are identified by their position in the event schema. Each encoded argument begins with a one-byte index, followed by its payload. Fixed-width values are written directly, whereas variable-length values include a four-byte length before their content. Strings and argument arrays use the same principle with bounded internal records. The wire format does not need a separate type tag because the index tells the Go event registry which name and decoder to use. It also permits an unavailable argument to be omitted without renumbering the arguments that follow it.

Before writing an argument, the eBPF helper must prove to the verifier that the current offset and the complete payload remain inside the buffer. Listing 4.8 shows the essential checks.

```
1  if (size == 0 || size > MAX_ELEMENT_SIZE || buf->offset >=
    ↪ ARGS_BUF_SIZE)
2      return 0;
3  u32 offset = buf->offset;
4  update_min(offset, ARGS_BUF_SIZE - 1);
5  u32 buffer_index_offset = offset + 1;
6  if (buffer_index_offset > ARGS_BUF_SIZE)
7      return 0;
8  buf->args[offset] = index;
9  // The omitted block clamps size for the older verifier.
10 u32 buffer_arg_offset = buffer_index_offset + size;
11 if (buffer_arg_offset > ARGS_BUF_SIZE)
12     return 0;
13 if (bpf_probe_read(&(buf->args[buffer_index_offset]), size, ptr) < 0)
14     return 0;
15 buf->offset = buffer_arg_offset;
16 buf->argnum++;
17 return 1;
```

Listing 4.8: Checked argument-buffer offset handling.

The current helper checks the original offset and uses `update_min` only to

make its upper bound visible to the verifier. It advances the stored offset only after the index and complete payload have been copied successfully.

Once the event is complete, the common submission helper refreshes its task context, optionally adds a user-space stack identifier and calculates the actual record size. It then sends the record through `bpf_perf_event_output` to the perf-event buffer associated with the current CPU. The Go runtime consumes these per-CPU buffers through a libbpfgo perf reader; the current implementation does not retain a BPF ring-buffer transport path.

Transport loss and decoding failure remain separate conditions. The reader's loss channel reports records dropped because a per-CPU buffer could not retain them, whereas malformed records that reach user space are reported by the decoder. Section 3.4.2 discusses the capacity and ordering consequences of this transport model.

4.4 Compatibility and Verifier Challenges

CO-RE adapts field accesses when compatible BTF information is available, but it does not remove every difference between kernel versions. The shared helpers therefore contain explicit alternatives for task identifiers and start times, capability layouts and inode change times. Cgroup identification also includes a dedicated path for the internal layout used by the RHEL 8 kernel family. At load time, CO-RE selects only branches whose required fields exist on the target kernel.

Some compatibility paths depend on more than structure layout. Syscall registers are read through BTF-aware task helpers when the kernel provides them. Otherwise, an x86-64-specific fallback derives the register frame from the task stack using the page and stack dimensions expected on the target system. Compatible 32-bit processes are detected separately and their syscall numbers are translated before event classification. The source also retains a bounded red-black-tree VMA lookup for kernels that use that representation. It is not called by the memory hooks described in Section 4.2.5 and therefore does not extend the evidence emitted by those events.

The verifier introduces a second class of constraints. Path reconstruction, string-array capture and VMA traversal use fixed loop bounds. Buffer writes and submitted record sizes repeat explicit checks in forms the target verifier can prove safe. The shared syscall-exit path also limits the return-argument index to values below 16, because the broader generic bound was rejected

on the target Rocky Linux kernel. Selected memory syscalls use dedicated typed tracepoints instead of the more complex generic dispatcher for the same reason.

These measures allow one eBPF object to accommodate several compatible kernel layouts while remaining acceptable to the verified target. They cannot create missing symbols, hooks, helpers or attachment mechanisms. The portability claim is therefore limited to the identified Rocky Linux environment, rather than to arbitrary Linux versions, consistently with the CO-RE boundaries introduced in Section 2.2.5.

4.5 Kernel-Space Implementation Summary

The kernel-space implementation combines direct, paired and lifecycle producers within one bounded event-construction framework. The hook families preserve the distinction between process requests, security mediation, committed transitions and returned outcomes, while the indexed argument contract allows unavailable fields to be omitted without changing their schema identity. Records cross the kernel boundary through the perf buffer after optional exact-UID admission.

The resulting object is intentionally adapted to the verified Rocky Linux target. CO-RE handles compatible layout differences, while explicit branches and verifier-visible bounds address older and vendor-specific kernel details. Chapter 5 now takes ownership of the user-space runtime that selects and loads these producers, decodes their records and evaluates them through policies and detectors.

Chapter 5

User-Space Runtime and Detection Implementation

Chapter 4 described how the eBPF programs construct and submit binary event records through the perf buffer. This chapter starts at the corresponding user-space boundary. It explains how the Go runtime turns the selected monitoring configuration into a managed user-space pipeline, from program preparation and record interpretation to policy-driven analysis, detector evaluation, and the delivery of monitoring and diagnostic results. The focus is on the implemented ownership and failure boundaries rather than on eBPF theory, hook catalogues or experimental results.

5.1 Loader, Probe Registry and Attachment Lifecycle

The runtime begins with one central configuration object populated by the command-line interface. Configuration normalisation enables the detector layer when detector paths are supplied and enables alert output for the `alerts-only` mode. Static validation then checks the requested output and logging formats before the runtime attempts to open an eBPF object or acquire a kernel-facing resource. The `list-events` path terminates at this stage: it obtains names from the probe registry and does not resolve BTF, open the eBPF object or attach a program.

The object and BTF inputs are resolved independently. For the eBPF ob-

ject, the runtime gives precedence to the `PROJECT_BPF_OBJECT` environment variable, then to an explicit command-line path, and finally to the object embedded in the executable. BTF follows the analogous `PROJECT_BTF_FILE`, command-line and default-host-path order. The embedded object removes the need to place a second object file beside the executable in the usual path, but it does not remove the target dependence on kernel BTF, privileges, compatible attachment points and the libraries used by the CGO-linked runtime. The latter uses `libbpfgo` as its Go interface to `libbpf` [3].

Policies and detector definitions are prepared before the eBPF module is opened. Policy files are loaded and checked against the event registry. When every active non-suppressive rule contains an explicit event include set, the policy manager computes a finite allowlist and intersects it with the optional CLI selection. The resulting set can reduce the programs considered for loading. An open-ended policy rule cannot safely produce such an allowlist, so the broader user selection is retained. A disjoint exact policy and CLI scope is rejected during startup rather than being misinterpreted as the empty selection that means “all events” to the probe selector.

Detector files are likewise parsed and validated before instances are created. Policies may select detector definitions by identifier or metadata, including severity, tags, statefulness and MITRE ATT&CK tactic or technique identifiers. This is a bootstrap operation: definitions outside the selected catalogue do not create detector instances or retain collective state. Detector selection does not, however, expand the eBPF event plan. A detector whose inputs are outside a general CLI selection will receive no such event; policy-selected detectors are checked against the selecting policy’s event scope.

The probe registry translates the selected logical event names into physical programs, hooks and attachment kinds. A public logical event may require several producers, for example different syscall variants, while an internal support program may be attached only because another event requires it. The logical set remains the user-facing selection and admission domain; the physical slice controls autoloading and attachment. Listing 5.1 shows the dependency expansion used to build the latter.

```
1 selectedProbes := make([]Spec, 0, len(supportedProbes))
2 for _, probe := range supportedProbes {
3     if _, ok := probeActivation[probe.eventName]; ok {
4         selectedProbes = append(selectedProbes, probe)
5         continue
6     }
7     for _, trigger := range probe.impliedBy {
```

```
8         if _, ok := probeActivation[trigger]; ok {
9             selectedProbes = append(selectedProbes, probe)
10            break
11        }
12    }
13 }
```

Listing 5.1: Physical probe-plan construction from selected logical events (abridged).

The first branch retains a producer directly selected by its logical event. The second retains a support program when one of its triggering events is active. Consequently, internal dependencies are available to the kernel-side implementation without becoming CLI-visible events or independent decoder schemas. Cross-registry tests exercise the public event set, dependency expansion and the many-to-one relation between physical programs and logical events. They are intended to detect interface drift; they do not establish the availability of every attachment on a particular target kernel.

Before the module is loaded, the runtime disables autoload for unselected programs represented in the registry. This ordering means that relocation, verification and loading apply only to the selected registered subset and its dependencies. The qualification is important: the registry controls registered program names; it is not an object-wide discovery mechanism for every program that might be present in the compiled eBPF object.

After loading, the runtime updates the singleton configuration map used by the optional exact effective-UID filter described in Chapter 4. It then creates one libbpfgo perf reader for the `events` map, with separate channels for records and reported losses, and attaches each selected program through the registry’s attachment route. Raw tracepoints, typed tracepoints, kprobes, kretprobes and the conditionally required cgroup attachment each use an explicit libbpfgo operation. The runtime retains every returned link.

Resource ownership is centralised in the runtime object. A signal-derived context stops the event loop cooperatively, while a deferred cleanup routine destroys links in reverse attachment order, releases only a cgroup-v2 mount created by the runtime, closes the perf reader and closes the module. Setup and attachment failures abort initialisation. By contrast, record-local decoding, detector and presentation failures are reported during the active loop and do not terminate later event processing. Cleanup errors are joined with an existing runtime error and reported through the diagnostic logger. The current tests strongly cover selection and configuration behaviour, whereas

object-backed tests for partial initialisation, attachment and cancellation remain a limitation.

5.2 User-Space Decoder and Event Registry

The perf reader delivers byte slices rather than self-describing event objects. The decoder is therefore the normalisation boundary for host events. It reads the fixed 136-byte common context using explicit little-endian offsets, obtains the declared argument count, resolves the numeric event identifier, and returns one typed `bufferdecoder.Event`. This representation contains the context, the public logical-event name and the decoded arguments; it is the same representation later used by policy matching and detector dispatch.

Explicit offset decoding avoids relying on Go structure padding. It also makes the cross-language contract visible: the field order, widths and total size must remain aligned with the C definition discussed in Section 4.3. The implementation does not generate this contract from one source or enforce it with a compile-time C/Go layout assertion, so changes to either side still require review and tests.

Argument interpretation is schema directed. The wire stream contains an index followed by a payload, but no independent type tag. The event registry maps the event identifier to a public name and ordered argument specification; the index then determines the expected name and decoding type. Listing 5.2 shows the central part of this procedure.

```
1 spec, ok := events.GetSpec(eventID)
2 if !ok {
3     return decodeUnknownArguments(rawArgs, eventID, argnum)
4 }
5 for seen := 0; seen < argnum && offset < len(rawArgs); seen++ {
6     index := int(rawArgs[offset])
7     offset++
8     if index >= len(spec.Args) {
9         return nil, fmt.Errorf("argument index %d out of range", index
10    ↪ )
11    }
12    argSpec := spec.Args[index]
13    value, consumed, err := decodeArgRecord(rawArgs[offset:], argSpec.
14    ↪ DecodeAs)
15    if err != nil {
16        return nil, err
17    }
```

```
16     offset += consumed
17     decoded[index] = Argument{Name: argSpec.Name, Type: argSpec.
    ↪ DecodeAs, Value: value}
18 }
```

Listing 5.2: Schema-directed decoding of indexed arguments (abridged).

Fixed-width scalar values, length-prefixed strings and byte sequences, bounded string arrays, compact argument arrays, credentials and socket-address records use dedicated payload decoders. Strings have terminal NUL bytes removed and must be valid UTF-8. Missing arguments are intentional: an eBPF producer may be unable to collect one field while preserving the schema indexes of fields that follow it. The decoder finally emits present arguments in schema order, rather than in their incidental order in the delivered record.

Two registries divide responsibility. The event registry is authoritative for the user-space public event identifiers, names and argument schemas. The probe registry is authoritative for which physical programs can be selected and attached. Several probes can converge on one schema, and internal support probes need not define a public decoded event. Cross-registry tests are used to detect drift between these domains, while retaining this separation of decoding and attachment concerns.

Table 5.1 states the decoder boundary precisely. It should not be described as a fully canonical wire parser: its purpose is bounded, schema-directed interpretation of records that reach user space.

Condition	Current behaviour	Boundary
Short common context, missing count, unknown event ID, invalid index, or truncated scalar/payload	Decoding fails and the record is discarded.	No policy, detector or output consumer receives the record.
Oversized variable value or invalid UTF-8 string	Decoding fails against configured size and text bounds.	Bounded payload interpretation.
Schema argument absent	Accepted; only encountered arguments are returned.	Intentional sparse encoding.
Duplicate index, trailing bytes, or a count that exceeds complete records at the end of the stream	Not universally rejected; a later duplicate replaces the earlier value.	The format is not strict canonical framing.
Producer type differs from the registered type at the same width	Not independently detectable because the wire format has no type tag.	Manual C/Go schema synchronisation remains necessary.

Table 5.1: Decoder admission and permissiveness boundaries.

Malformed records and transport loss are distinct. A loss notification means that no record bytes were retained by the perf buffer; a decode error means that bytes reached user space but could not be safely interpreted. Both are diagnosed without converting the condition into a monitored event, and later records continue through the loop.

5.3 User-Space Event Selection and Admission

After decoding, the runtime applies three short-circuit admission gates. The first checks whether the decoded logical name belongs to the selected event set. The second applies an optional exact command-name filter. The third asks the policy manager whether the event is selected by the active policy set. A policy evaluates its common scope and then its rules; matching suppress rules override monitor or detect rules. With no configured policies, this final gate is permissive.

Listing 5.3 shows the admission order and the resulting fan-out. Diagnostics and optional socket enrichment are omitted because they do not change the host-event admission contract.

```
1 event, err := bufferdecoder.DecodeEvent(raw)
2 if err != nil {
3     return
4 }
5 if !p.eventEnabled(event.EventName) {
6     return
7 }
8 if !p.commEnabled(event.Context.CommString()) {
9     return
10 }
11 if !p.policyEnabled(event) {
12     return
13 }
14 if !p.cfg.Alerts.Only {
15     if err := p.printer.Print(event); err != nil {
16         p.logger.Error("print event", zap.Error(err))
17     }
18 }
19 p.processDetectors(ctx, event)
```

Listing 5.3: Decoded-event admission and consumer fan-out (abridged).

An admission failure prevents both normalised-event presentation and detector evaluation. The **alerts-only** option has a narrower role: it suppresses only normalised-event presentation after admission. It does not bypass policy matching, disable detector processing or suppress alert presentation. This keeps collection and analytical semantics independent from whether an operator wants to see the ordinary event stream.

The policy manager is intentionally not a detector. Policies decide which observations are relevant to a scenario through event, command, UID and argument constraints; detectors decide whether one admitted event or a short sequence produces an alert. The current policy modes record monitor, detect or suppress intent, but monitor and detect matches share the same admission path. Suppression is the only mode that removes a matching event from subsequent consumers.

The active loop distinguishes valid rejections from failures. At debug level, only the first few filter drops are logged in order to avoid unbounded diagnostic output. Decode errors, detector errors and rendering errors are logged separately, while the loop remains active. This behaviour is established by

code inspection and component tests; a direct end-to-end unit test for the complete admission order is still absent.

5.4 Policy and Detector Engine

5.4.1 YAML Policy and Detector Definitions

Policies and detectors enter the runtime through separate YAML models. A policy contains a name, mode, intent, common scope and one or more rules. Empty policy mode and intent default to `monitor` and `threat_hunting`; an event selector with no include list means all events except explicit exclusions. Rules compose as alternatives within a policy, whereas the fields of one rule compose as constraints. A suppress policy may not select detectors.

Detector documents provide stable identity and descriptive metadata together with consumed events, conditions, optional ordered steps, output information, and optional ATT&CK metadata. The loader reads explicit files or the YAML files in supplied directories in deterministic lexical order, validates known event names, and compiles conditions, numeric constants, event sets and correlation plans before event processing begins. The current YAML decoder is not strict about unknown fields; a spelling error can therefore be ignored by the external parser rather than becoming a schema error. This is an implementation limitation rather than a feature of the rule language.

Policy metadata can reduce the loaded detector catalogue using identifiers, severity, tags, statefulness, tactics and techniques. Values within one criterion have alternative semantics, while populated criteria are conjunctive. Selections from separate policies are combined. The complete detector catalogue is parsed before this selection, so an invalid unselected definition can still fail startup. The selection check currently uses the broad event-name set of a definition, including optional requirements; Chapter 6 should evaluate the practical consequences of this conservative choice rather than treating it as a claim of detection coverage.

5.4.2 Point and Collective Detection

The detector engine creates a validated registry and an event-name dispatch index once at bootstrap. The index prevents each decoded event from being

offered to the entire catalogue. Listing 5.4 shows the event-time dispatch loop. It invokes only candidates for the current event and keeps one detector failure from preventing the remaining candidates from running.

```
1 for _, detector := range detectors {
2     result.Invoked++
3     alerts, err := detector.OnEvent(ctx, event)
4     if err != nil {
5         result.Errors = append(result.Errors, DetectorError{
6             DetectorID: detector.Definition().ID,
7             Err:        err,
8         })
9         continue
10    }
11    if len(alerts) > 0 {
12        result.Alerts = append(result.Alerts, alerts...)
13    }
14 }
```

Listing 5.4: Event-indexed detector dispatch with isolated errors.

A point detector applies its compiled top-level conditions to one admitted event. If it declares steps, the event must also match a step and its step conditions. A successful point match immediately creates an alert containing that source event. Conditions support equality, inequality, containment, membership, prefix and suffix tests, existence tests, and numeric comparisons. The implementation therefore performs deterministic rule evaluation; it does not train on observed activity or calculate an anomaly score.

A collective detector is a stateful detector with more than one ordered step. It first applies its global conditions, derives local correlation keys, and attempts to advance the next expected step of a partial sequence. Completion creates an alert containing the ordered evidence. An event that matches the first step can start or replace the partial sequence for its key. Events need not be contiguous in the complete input stream, but they are considered in the arrival order delivered to the runtime. The engine does not reorder records or infer cross-CPU causality.

5.4.3 Local Correlation Strategies, State Bounds and Alert Deduplication

Correlation plans are compiled during detector construction, so the hot path does not parse strategy names from YAML. Process correlation uses the host

PID together with task start time, preventing a reused numeric PID from joining two process lifetimes. Process-tree correlation additionally tries the immediate parent identity for a later event; it does not reconstruct arbitrary ancestry or maintain a persistent process graph. Resource correlation uses the device–inode pair in compatible event arguments, not the pathname. Cgroup correlation uses the local kernel cgroup ID. Composite correlation concatenates the configured dimensions and requires every component to be present and equal. These values are local observation keys, not Kubernetes workload identities.

The unavailable `user_session` strategy is rejected at parsing time because normalised events do not expose a stable local session identifier. The implementation deliberately does not approximate a session with a UID. Missing process, resource or cgroup data yields no correlation key and therefore no state transition; it does not fall back to a weaker identity.

Listing 5.5 shows the core transition of a collective detector. The omitted code constructs the initial key and stores the first matching event; it does not alter the one-state-per-key rule.

```
1  for _, key := range lookupKeys {
2      state, ok := d.state[key]
3      if !ok {
4          continue
5      }
6      if now.Sub(state.firstSeen) > window {
7          delete(d.state, key)
8          continue
9      }
10     matches, err := stepMatches(event, d.steps[state.nextStep])
11     if err != nil {
12         return nil, err
13     }
14     if matches {
15         state.events = append(state.events, event)
16         state.nextStep++
17         if state.nextStep >= len(d.steps) {
18             delete(d.state, key)
19             return []detectors.Alert{d.newAlertFromEvents(state.events
20         ⇨ )}, nil
21         }
22         d.state[key] = state
23         return nil, nil
24     }
```

24 }

Listing 5.5: Advancing one bounded collective sequence (abridged).

Table 5.2 contains the local-state limits in one place. They are safeguards for a constrained monitoring agent, not performance results. Expiry is checked for the key addressed by a relevant event and the detector periodically prunes unrelated state while it processes qualifying traffic. Although the engine exposes `Flush`, the production runtime does not call it on an independent timer. Idle stale state is consequently removed on later relevant traffic, explicit API use, or capacity pressure.

Mechanism	Current value	Effect
Default stateful window	2 seconds	Used when a stateful detector omits an explicit duration.
Maximum stateful window	5 seconds	Longer definitions fail validation; the bound limits retained partial evidence.
State pruning interval	$\min(\text{window}/2, 1 \text{ second})$	Avoids a full scan for every relevant event while retaining direct expiry checks for the addressed key.
Partial sequence capacity	4,096 keys per detector	A new key at capacity evicts the oldest incomplete state; an existing key is updated.
Alert deduplication	5-second window; entries older than 10 seconds are pruned	Suppresses repeated alert keys after matching. The cache has no independent capacity bound.

Table 5.2: Local-correlation and alert-deduplication bounds.

Alert deduplication is deliberately separate from collective correlation. It operates after a detector has produced an alert, using detector identity, title, severity and fields from the first evidence event. It suppresses repeated equivalent keys within the configured processing-time window, but does not represent the complete collective sequence, the correlation key or a durable causal identity. It is therefore a presentation-noise control rather than a second correlation mechanism.

5.4.4 MITRE ATT&CK Metadata Propagation

Detector definitions may contain a framework, version, tactics, techniques, data sources and data components. The implementation validates the syntactic shape of tactic and technique identifiers and copies this metadata into each matching alert. Policies may also use tactic and technique identifiers to select detector definitions during bootstrap. This gives alert consumers a consistent MITRE ATT&CK vocabulary for classifying the intended security meaning of a detector [28].

The local runtime does not download an ATT&CK catalogue or validate whether a mapping is semantically appropriate. Unmapped detectors are accepted. ATT&CK metadata should consequently be presented as classification and traceability information attached to detector evidence, not as proof of detector correctness, technique coverage or research novelty.

5.5 Output and Structured Logging

The output package is a presentation boundary, not another normalisation stage. It adapts the decoded event into JSON or compact text and adapts a detector alert into a user-facing record containing detector identity, severity, ordered evidence and optional threat metadata. JSON preserves typed values and may add readable labels, whereas compact text is intended for interactive inspection. The event model used by output is derived from the same decoded representation used by policies and detectors.

Normalised event output and alert output both use standard output. They are configured independently and can therefore use different formats in one run; the resulting stream is not guaranteed to be one homogeneous machine-readable schema. The `alerts-only` option suppresses normalised event output but leaves detector evaluation and alert rendering active. Point alerts expose one source event, while collective alerts retain their ordered evidence in JSON and render a bounded sequence in compact text.

Operational diagnostics follow a separate path. The Zap logger writes to standard error in console or JSON form, with debug, info, warn and error levels. It receives runtime lifecycle messages, selected libbpf diagnostics, perf-buffer loss notifications, decoding failures, detector failures and presentation failures [57]. Zap does not serialise monitored events or alerts. This prevents infrastructure errors from being mistaken for security observations

and allows diagnostics to change without changing the data model seen by event and alert consumers.

Table 5.3 summarises the operating channels and error boundaries. Fatal configuration and startup failures are returned to the command entry point, while most record-local failures are logged and the loop continues. Logger synchronisation errors are currently discarded, so the design does not claim durable delivery of every diagnostic message.

Class	Source model	Channel	Failure behaviour
Normalised event	<code>bufferdecoder.Event</code> through a printer	stdout	A write failure is diagnosed; subsequent events continue.
Detector alert	Detector alert with one event or ordered evidence	stdout	A rendering failure is diagnosed; it does not change detector state.
Operational diagnostic	Runtime, decoder, detector, output or libbpf condition	stderr through Zap	Record-local conditions are logged without becoming security events.
Fatal bootstrap or lifecycle error	Command, definition loading or runtime setup return value	stderr through the command path	Startup or shutdown reports an error; cleanup errors are joined where available.

Table 5.3: User-space output and diagnostic boundaries.

5.6 Containerisation and Deployment Considerations

The container build follows a builder/runtime split. The builder stage compiles the local libbpf dependency, eBPF object and Go executable; the runtime stage copies the resulting executable and declares the runtime packages used by that image. As described in Chapter 4, the eBPF object is embedded during the Go build, so the normal executable path does not require a neighbouring object file.

The image does not embed or copy the policy and detector YAML catalogues. Deployments that use these inputs must supply them explicitly, for example through a bind mount or a derived image. Similarly, BTF and the

relevant kernel facilities remain host resources. The provided Docker path uses broad host integration and privileges sufficient for the prototype; it is not a minimal-privilege security profile.

No Kubernetes manifest, lifecycle configuration, least-privilege capability set or deployment validation is implemented in the repository. A future orchestration layer may place the monitoring agent in a wider platform, but this chapter makes no claim that such a deployment has been implemented or evaluated.

5.7 User-Space Implementation Summary

The user-space implementation turns a selected perf-buffer record into a typed event, applies explicit admission gates, and then routes admitted evidence to presentation and deterministic detection. Policies control scope and suppression, while detectors evaluate point conditions or bounded local sequences and attach ATT&CK classification metadata to their alerts. The runtime keeps kernel-facing resource ownership, record interpretation, analytical state and presentation responsibilities separate.

The implementation also defines clear boundaries for the following evaluation. The registry and decoder tests establish important structural invariants, but attachment behaviour, output under workload, alert quality, loss and resource consumption require target-environment evidence. Chapter 6 evaluates these behaviours on the stated Rocky Linux environment.

Appendix A

Appendix A

Appendix text here.

Appendix B

Appendix B

Appendix text here.

Bibliography

- [1] Jagrati Agrawal et al. “Efficient Pattern Matching over Event Streams”. In: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. Ed. by Jason Tsong-Li Wang. New York, NY, USA: Association for Computing Machinery, 2008, pp. 147–160. ISBN: 978-1-60558-102-6. DOI: [10.1145/1376616.1376634](https://doi.org/10.1145/1376616.1376634).
- [2] Aqua Security. *Installing Tracee*. Official Tracee documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/install/> (visited on 07/29/2026).
- [3] Aqua Security. *libbpfgo: An eBPF Library for Go Powered by libbpf*. Official project repository. 2026. URL: <https://github.com/aquasecurity/libbpfgo> (visited on 07/27/2026).
- [4] Aqua Security. *Tracee Architecture*. Official architecture documentation for Tracee version 0.15. 2023. URL: <https://aquasecurity.github.io/tracee/v0.15/contributing/architecture/> (visited on 07/29/2026).
- [5] Aqua Security. *Tracee Custom Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/custom/overview/> (visited on 07/29/2026).
- [6] Aqua Security. *Tracee Detector API Reference*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/api-reference/> (visited on 07/27/2026).
- [7] Aqua Security. *Tracee Detectors*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/> (visited on 07/27/2026).
- [8] Aqua Security. *Tracee Documentation*. 2026. URL: <https://aquasecurity.github.io/tracee/latest/> (visited on 07/17/2026).

BIBLIOGRAPHY

- [9] Aqua Security. *Tracee Documentation Overview*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/overview/> (visited on 07/29/2026).
- [10] Aqua Security. *Tracee Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/> (visited on 07/27/2026).
- [11] Aqua Security. *Tracee Output Configuration*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/outputs/> (visited on 07/27/2026).
- [12] Aqua Security. *Tracee Policies: Overview*. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/> (visited on 07/17/2026).
- [13] Aqua Security. *Tracee Policy Rules*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/rules/> (visited on 07/29/2026).
- [14] Aqua Security. *Tracee Security Model*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/security-model/> (visited on 07/29/2026).
- [15] Varun Chandola, Arindam Banerjee, and Vipin Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys* 41.3 (July 2009), 15:1–15:58. DOI: [10.1145/1541880.1541882](https://doi.org/10.1145/1541880.1541882).
- [16] Gianpaolo Cugola and Alessandro Margara. “Processing Flows of Information: From Data Stream to Complex Event Processing”. In: *ACM Computing Surveys* 44.3 (June 2012), 15:1–15:62. DOI: [10.1145/2187671.2187677](https://doi.org/10.1145/2187671.2187677).
- [17] Hojat Gazestani. *Understanding Linux Process UIDs: Real, Effective, Saved, and Filesystem*. Sept. 2, 2025. URL: <https://hojat-gazestani.github.io/blogs/Linux-Standard-UNIX-Access-Control> (visited on 08/04/2026).
- [18] Bolaji Gbadamosi et al. “The eBPF Runtime in the Linux Kernel”. In: *arXiv preprint arXiv:2410.00026* (2024). DOI: [10.48550/arXiv.2410.00026](https://doi.org/10.48550/arXiv.2410.00026). URL: <https://arxiv.org/abs/2410.00026>.
- [19] Elazar Gershuni et al. “Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions”. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Ed. by Kathryn S. McKinley and Kathleen Fisher. PLDI ’19. Association for Computing Machinery, 2019, pp. 1069–1084. DOI:

BIBLIOGRAPHY

- 10.1145/3314221.3314590. URL: <https://doi.org/10.1145/3314221.3314590>.
- [20] Yi He et al. “Cross Container Attacks: The Bewildered eBPF on Clouds”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Ed. by Joseph A. Calandrino and Carmela Troncoso. Anaheim, CA: USENIX Association, Aug. 2023, pp. 5971–5988. ISBN: 978-1-939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/he>.
- [21] Jin Her et al. “An In-Depth Analysis of eBPF-Based System Security Tools in Cloud-Native Environments”. In: *IEEE Access* 13 (2025), pp. 155588–155604. DOI: 10.1109/ACCESS.2025.3605432. URL: <https://ieeexplore.ieee.org/document/11146725/>.
- [22] Linux Kernel Community. *eBPF Userspace API*. Linux kernel 6.9 documentation. 2024. URL: <https://www.kernel.org/doc/html/v6.9/userspace-api/ebpf/index.html> (visited on 07/17/2026).
- [23] Linux Kernel Community. *eBPF Verifier*. 2026. URL: <https://docs.kernel.org/bpf/verifier.html> (visited on 07/17/2026).
- [24] Linux man-pages project. *bpf-helpers(7) — List of eBPF Helper Functions*. 2026. URL: <https://man7.org/linux/man-pages/man7/bpf-helpers.7.html> (visited on 07/27/2026).
- [25] Linux man-pages project. *bpf(2) — Perform a Command on an Extended BPF Map or Program*. 2026. URL: <https://man7.org/linux/man-pages/man2/bpf.2.html> (visited on 07/27/2026).
- [26] Linux man-pages project. *inode(7) — Linux Manual Page*. 2026. URL: <https://man7.org/linux/man-pages/man7/inode.7.html> (visited on 08/06/2026).
- [27] Red Hat. *What Is Backporting and How Does It Affect Red Hat Enterprise Linux?* Red Hat Customer Portal. June 2025. URL: <https://access.redhat.com/solutions/57665> (visited on 07/27/2026).
- [28] Blake E. Strom et al. *MITRE ATT&CK: Design and Philosophy*. Tech. rep. The MITRE Corporation, Mar. 2020. URL: <https://www.mitre.org/news-insights/publication/mitre-attck-design-and-philosophy>.
- [29] Dave Thaler. *BPF Instruction Set Architecture (ISA)*. RFC 9669. Internet Engineering Task Force, Oct. 2024. DOI: 10.17487/RFC9669. URL: <https://www.rfc-editor.org/info/rfc9669>.

BIBLIOGRAPHY

- [30] The Falco Project Authors. *Falco Event Sources*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/> (visited on 07/27/2026).
- [31] The Falco Project Authors. *Falco Kernel Event Architecture*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/kernel/architecture/> (visited on 07/27/2026).
- [32] The Falco Project Authors. *Falco Outputs*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/outputs/> (visited on 07/27/2026).
- [33] The Falco Project Authors. *Falco Rule Conditions*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/conditions/> (visited on 07/27/2026).
- [34] The Falco Project Authors. *Falco Rules*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/> (visited on 07/27/2026).
- [35] The Linux Kernel Documentation. *BPF Design Q&A*. 2026. URL: https://docs.kernel.org/bpf/bpf_design_QA.html (visited on 07/27/2026).
- [36] The Linux Kernel Documentation. *BPF LLVM Relocations*. 2026. URL: https://docs.kernel.org/bpf/llvm_reloc.html (visited on 07/27/2026).
- [37] The Linux Kernel Documentation. *BPF Maps*. 2026. URL: <https://docs.kernel.org/bpf/maps.html> (visited on 07/27/2026).
- [38] The Linux Kernel Documentation. *BPF Ring Buffer*. 2026. URL: <https://docs.kernel.org/bpf/ringbuf.html> (visited on 07/27/2026).
- [39] The Linux Kernel Documentation. *BPF Ring Buffer*. 2026. URL: <https://docs.kernel.org/bpf/ringbuf.html> (visited on 08/03/2026).
- [40] The Linux Kernel Documentation. *BPF Type Format (BTF)*. 2026. URL: <https://docs.kernel.org/bpf/btf.html> (visited on 07/27/2026).
- [41] The Linux Kernel Documentation. *Classic BPF vs eBPF*. Describes execution through architecture-specific JIT compilation and, when JIT compilation is not used, through the `__bpf_prog_run()` interpreter. URL: https://docs.kernel.org/bpf/classic_vs_extended.html (visited on 07/27/2026).
- [42] The Linux Kernel Documentation. *Documentation for /proc/sys/net/: BPF JIT Controls*. 2026. URL: <https://docs.kernel.org/admin-guide/sysctl/net.html#bpf-jit-enable> (visited on 07/27/2026).

BIBLIOGRAPHY

- [43] The Linux Kernel Documentation. *Kernel Probes (Kprobes)*. 2026. URL: <https://docs.kernel.org/trace/kprobes.html> (visited on 07/27/2026).
- [44] The Linux Kernel Documentation. *libbpf Overview*. 2026. URL: https://docs.kernel.org/bpf/libbpf/libbpf_overview.html (visited on 07/27/2026).
- [45] The Linux Kernel Documentation. *Linux Security Module Development*. 2026. URL: <https://docs.kernel.org/security/lsm-development.html> (visited on 07/27/2026).
- [46] The Linux Kernel Documentation. *Linux Tracing Technologies*. 2026. URL: <https://docs.kernel.org/trace/index.html> (visited on 07/27/2026).
- [47] The Linux Kernel Documentation. *Pathname Lookup*. 2026. URL: <https://docs.kernel.org/filesystems/path-lookup.html> (visited on 08/06/2026).
- [48] The Linux Kernel Documentation. *Perf Ring Buffer*. 2026. URL: https://docs.kernel.org/userspace-api/perf_ring_buffer.html (visited on 07/27/2026).
- [49] The Linux Kernel Documentation. *Program Types and ELF Sections*. 2026. URL: https://docs.kernel.org/bpf/libbpf/program_types.html (visited on 07/27/2026).
- [50] The Linux Kernel Documentation. *The Linux Kernel API*. Version 6.15.0. Documentation for the `security_file_open` Linux Security Module hook. URL: https://docs.kernel.org/6.15/core-api/kernel-api.html#c.security_file_open (visited on 08/04/2026).
- [51] The Linux Kernel Documentation. *Using the Linux Kernel Tracepoints*. 2026. URL: <https://docs.kernel.org/trace/tracepoints.html> (visited on 07/27/2026).
- [52] The Tetragon Authors. *Tetragon Enforcement*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/enforcement/> (visited on 07/27/2026).
- [53] The Tetragon Authors. *Tetragon Events*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/events/> (visited on 07/27/2026).
- [54] The Tetragon Authors. *Tetragon Installation FAQ*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/installation/faq/> (visited on 07/27/2026).

BIBLIOGRAPHY

- [55] The Tetragon Authors. *Tetragon Tracing Policies*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/> (visited on 07/27/2026).
- [56] The Tetragon Authors. *Tetragon Tracing Policy Selectors*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/selectors/> (visited on 07/27/2026).
- [57] Uber Go. *Zap: Structured, Levelled Logging in Go*. URL: <https://github.com/uber-go/zap> (visited on 07/31/2026).

To my family and friends, for their support.